TECHNICAL DEEP-DIVE

Inaya Ecosystem — Developer Reference

Contract functions, API routes, data models, and SDK signatures — a lookup reference grounded directly in the real code.

Document INAYA-DEV-2026-V1 · Classification Internal · October 2026

SECTION 01

Repo & Entry-Point Map

PATH	ENTRY POINT
contracts/ (repo root)	*.sol — Hardhat project, hardhat.config.js at repo root
scripts/ (repo root)	deploy*.cjs / deploy*.js, verify-*.cjs, simulate-security-nodes.js
inaya-network-dapp/	Next.js 14 App Router — src/app/, src/lib/, src/app/api/
inaya-network-dapp/custody-sdk/	src/index.js (InayaKernel), src/crypto.js, src/contracts.js
inaya-network-dapp/custody-sdk/packages/node-daemon/	bin/inaya-node-daemon.js (commander CLI)
src/lib/chain-adapters/ (bridge + interop)	registry.js (capability model), interop/ (Wormhole)
src/lib/ai-action-requests.js, auditChain.js	AI guarded-execution state machine + cryptographic audit trail
inaya-mobile/	Expo — App.js, src/screens/, src/utils/, src/providers/
inaya-desktop/	src-tauri/src/lib.rs, tauri.conf.json
inaya-dapp-desktop/	src-tauri/src/lib.rs, tauri.conf.json

SECTION 02

Contract Function Reference

InayaStaking.sol

Constructor(stakingToken, rewardToken) — both = $INAYA address.

- stake(uint256 amount, uint256 lockPeriodDays) — lockPeriodDays ∈ {0,30,90}, multiplier 1.00x/1.25x/1.50x, locked in on first stake
- withdraw(uint256 amount) — reverts if block.timestamp < lockExpiry
- claimReward() / exit() (withdraw all + claim)
- earned(address) / rewardPerToken() / getUserTier() → "None"/"Standard"/"Enterprise Priority"
- owner-only: setRewardRate, fundRewardPool, setEnterpriseTierThreshold, recoverForeignToken (blocked on staking/reward token)
- events: Staked, Withdrawn, RewardPaid, RewardRateUpdated, RewardPoolFunded

InayaEgressTimelockVault.sol

Constructor(inayaToken, usdtToken, stakingContract, operationalTreasury).

- executeSemiAnnualHarvest() — public, callable by anyone once currentEpochStart + 180 days has passed; sweeps full INAYA balance to stakingContract via raw transfer (not fundRewardPool)
- forwardUSDT() — public, sweeps full USDT balance to operationalTreasury
- getNextHarvestCountdown() view
- owner-only: setStakingContract, setOperationalTreasury

InayaCorporateEscrow.sol

Constructor(usdtToken) only.

- createEscrow(address corporate, address node, uint256 totalAmount) — safeTransferFrom(caller) into escrow, MONTHS=12 fixed schedule
- releaseMonthlyPayout(uint256 scheduleId) — public, callable by anyone once due
- owner-only: setMonthSeconds (testnet-only timing override), reassignNode
- events: EscrowCreated, MonthlyReleased, EscrowClosed

InayaNodeRegistry.sol

Constructor(usdtToken, verifierWallet). SCOPE NOTE in header: metrics are coordinator-verified, NOT cryptographic proof-of-storage.

- registerNode(uint256 capacityGB) — self-serve
- updateNodeMetrics(...) — verifier-only
- queueSettlement / queueSettlementsBatch — verifier-only, computes commission (30/40/50% by Entry/Mid/Enterprise tier), does not move funds
- releaseSettlement / releaseSettlementsBatch — public, callable by anyone, only after SETTLEMENT_DELAY = 36 hours
- owner-only: setVerifierWallet, setTierThresholds, withdrawExcessReserve

InayaProofRegistry.sol

Constructor(custodyAddress) — immutable IInayaCustody reference.

- registerMerkleRoot(bytes32 fileHash, bytes32 merkleRoot, uint256 chunkCount, address node) — requires msg.sender == custody.assets(fileHash).owner; node must be pre-approved via setNodeRegistered
- verifyChunkProof(...) — onlyOwner today (backend-checked); header comment documents a planned permissionless + slashing path
- events: MerkleRootRegistered, ProofVerified, NodeRegistrationChanged

InayaCustody (interface only — IInayaCustody)

Deployed, source not tracked in this repo. Interface as referenced from InayaProofRegistry:

- assets(bytes32 fileHash) view → {owner, cidAlpha, cidBeta, size, timestamp}
- batchRegisterAssets(bytes32[] fileHashes, uint256[] fileSizes, string[] shardACIDs, string[] shardBCIDs)
- event AssetRegistered

RevenueRouter (interface only)

Deployed, source not tracked in this repo. Called via inline ABI from page.js and the Stripe webhook route:

- processCorporateInvoice(uint256 usdtAmount) external

Security Layer contracts

InayaThreatRegistry (status ledger, updated only by Reporter), InayaThreatReporter (confirmThreat(bytes32 threatId, uint8 category, uint16 confidenceBps, bytes32 contributingNodesHash) — relayer-only), InayaNodeReputation (checkpointReputation(...) — relayer-only, periodic), InayaSecurityPolicy (publishPolicy(uint256 version, bytes32 policyHash, string policyURI) — owner/relayer-only).

Oracle & Automation Layer contracts (deployed to BSC Testnet)

InayaOracleRegistry (registerSource(bytes32 id, string dataType, address submitter, uint256 updateFrequency) — owner-only; isAuthorizedSubmitter(id, address) view), InayaOracleAdapter (submitData(bytes32 id, uint256 value, uint256 reportedTimestamp) — reverts on future/stale timestamp, sub-minimum-interval, or excess deviation; getLatestData(id) / isStale(id) views), InayaAutomationRegistry (registerTask(bytes32 id, address target, bytes4 selector, string condition) — owner-only; recordExecution(id, bool success, uint256 nextEligible, bytes32 txHash) — worker/owner-only, pure audit-trail write, never forwards a call to target).

SECTION 03

custody-sdk (InayaKernel) API Reference

FUNCTION	PURPOSE
generateSecureSalt()	16-byte random salt via crypto.getRandomValues
deriveVaultKey({passkey, salt, iterations=100000, algo})	PBKDF2 → 32-byte AES key
disperseAndSlice({file, encryptionKey})	Encrypt (AES-GCM-256) + base64-midpoint-split into shardAlpha/shardBeta
anchorToLedger(...)	Calls InayaCustody.batchRegisterAssets() with CIDs (not raw shard bytes)
retrieveAndReconstruct({connection, fileHash, passkey, fetchShard})	Reads assets(fileHash), fetches both shards, reassembles
reconstructAndDecrypt({shardAlpha, shardBeta, passkey})	Concat → split salt/IV/ciphertext → re-derive key → AES-GCM decrypt
encryptForPublicKey / decryptWithSecretKey / deriveEncryptionKeypairFromSignature	X25519 + HKDF-SHA256 + XChaCha20-Poly1305 sealed-box — re-wraps the passkey for sharing, not the file
connectWallet, approveFeeTokens	ethers v6 wallet/allowance helpers
Staking.{stake,unstake,claimReward,calculateReward,getStakedBalance}	Thin wrapper over InayaStaking
Payments, Metadata, events (EventEmitter)	Supporting modules
createPasskeyBackup / restorePasskeyBackup / isPasskeyBackupEnvelope	Export/restore a passkey-derived vault as an encrypted, portable backup file (since v1.0.6-beta)
TrustPlatform.{verifyAudit,getEvidence,checkPermission}	Client for the public trust/evidence API — see Section 21
IdentityIntegration.{signWebhook,sendEvent}	Client for the Identity Integration webhook API — see Section 36
ComplianceProofs.{proveAuthorizedAccess,verifyAuthorizedAccessProof}	Trust Fabric SOW: real Schnorr/Fiat-Shamir zero-knowledge proof of key ownership bound to {policy, action, timestamp} — production-ready, wired into InayaKernel
Intent.{create,discoverRoutes,simulate,riskCheck,execute}	Trust Fabric SOW: validate/discover/simulate/risk-check/execute pipeline in front of the real bridge SDK; a route-substitution vulnerability was found and fixed during adversarial testing
Attestation.{verify,evaluatePolicy}	Trust Fabric SOW: device/environment attestation verification against a policy — a fully-tested software-simulated TEE pipeline, honestly labeled non-hardware-backed
Analytics.getWalletStorageStats()	Real per-wallet storage usage/cost statistics
Backup.{getBackupStatus,getBackupHealth,getRedundancyStatus,requestRecovery}	Shard redundancy/recovery status client
AppStore.{submitListing,getListings,getMyListings}	Web3 App Store listing client
errors. {InayaError,InayaValidationError,InayaWalletError,InayaContractError,InayaNetworkError}	Typed error classes

Crypto primitives are @noble/hashes, @noble/ciphers, @noble/curves (pure JS) rather than crypto.subtle — chosen specifically for React Native compatibility, per an inline comment in crypto.js.

Trust Fabric — 91/91 tests passing (55 new). ComplianceProofs, Intent and Attestation (Institutional Trust Infra SOW, added September 2026) are documented in full in custody-sdk/docs/trust-fabric-phase0-1.md and trust-fabric-phase3-adr.md.

Release verification (v1.0.10-beta+)

Every git tag matching v* on custody-sdk (.github/workflows/release.yml) runs npm test → computes checksums → pins to IPFS (Pinata, non-blocking) → npm publish --provenance → commits CHECKSUMS.md → attaches the tarball to a GitHub Release. Reproduce independently per docs/VERIFYING_RELEASES.md:

- git rev-parse <tag>^{tree} — compare to CHECKSUMS.md's git-tree-hash. NOT git archive | sha256sum (tried first, found non-reproducible across git versions — see Section 05 of ecosystem-architecture.js).
- Download the .tgz attached to the GitHub Release, sha256sum it, compare to CHECKSUMS.md's npm-tarball-sha256 — always matches, since you're hashing the literal file CI attached rather than re-deriving it (npm pack tarball bytes aren't guaranteed identical across npm versions, even when file contents are).
- npm view @inaya-network/custody-sdk@<version> dist.integrity / dist.shasum — the registry's own recorded hash for the published tarball.
- test/webCryptoCompat.test.mjs — the committed proof that this package's crypto and the dApp's former inline crypto.subtle implementation are byte-identical and cross-decryptable.

SECTION 04

node-daemon CLI Reference

COMMAND	EFFECT
login	Reads INAYA_PRIVATE_KEY (env or prompt), encrypts at rest (PBKDF2-SHA256 200k iter + AES-GCM-256), writes ~/.inaya/node-daemon/config.json
register <capacityGB>	InayaNodeRegistry.registerNode(capacityGB) on-chain + POST /api/nodes/register off-chain
start	Foreground 5-min heartbeat loop → POST /api/nodes/heartbeat with {nodeId, operatorWallet, totalCapacityGB, usedCapacityGB:0, shardsStored:0} (last two hardcoded 0 — no shard-storage capability exists)
report <indicator> --category <cat>	Security Layer signed observation → POST /api/security/report
status	Shows what the coordinator has recorded about this node: heartbeat recency, uptime score, daemon version/restarts, and threat-reporting reputation
service install / uninstall	Windows background service via node-windows

> package.json description states it plainly: registers your node on-chain and reports heartbeat/telemetry. Does not store or serve shards. NODE_REGISTRY_ABI in this package has exactly two functions (registerNode, nodes) — no stake/settlement/withdraw calls exist in the daemon's on-chain surface.

SECTION 05

Backend lib/ Reference

Every file below follows the same shape: getXCollections(), ensureXIndexes() (module-level indexesEnsured guard), validateXInput() (throws, fail-closed).

FILE	DOMAIN
security.js	Threat reports/threats/policy/events/reputation-cache, computeThreatConfidence, getPublicSecurityStats
ai-security-tools.js	Security Assistant tool declarations + dispatcher
learn.js / learnConfig.js / youtube.js	Learn saved/progress/search-cache/video-cache, category+collection config, cached YouTube Data API client
ai-learn-tools.js	Learn Tutor tool declarations + dispatcher
orgs.js / orgPlans.js	Business Workspace auth (sessions, magic-links), org/plan definitions; canManageOrg/canAccessDepartment plus the additive canManageFinance/canAccessFinance/canManageHR/canAccessHR/isDepartmentManager/isSelfEmployeeRecord gates
document-workflow.js / document-permissions.js	Document state machine + activity log; access resolution + share tokens + getAccessibleScope() (the one org-wide 'what can this member see' resolver, extended per module as each shipped)
ai-business-tools.js	Business Assistant tool declarations + dispatcher (permission-aware) — list_documents/tasks/contacts/deals/suppliers/purchase_orders/products/invoices/expenses/employees, find_employee_document
task-workflow.js / deal-workflow.js / purchase-request-workflow.js / purchase-order-workflow.js	Business Operations transition-table workflows — {from,to,requiresManage,activityAction} pattern, one findOneAndUpdate per transition
inventory.js	recordStockMovement(), getStockLevel(), totalStockForProduct(), isLowStock() — stock_levels is $inc-only, stock_movements is the append-only ledger
invoice-workflow.js / expense-workflow.js	transitionInvoice() (send/markPaid/cancel + cron-only markOverdueInvoices()), transitionExpense() (submit/approve/reject/cancel)
employee-workflow.js / leave-workflow.js	transitionEmployee() (onboarding→active→on_leave→terminated), transitionLeaveRequest(), getLeaveBalance() (computed fresh, never a stored counter)
attachments.js	createAttachment()/listAttachmentsForRecord()/serializeAttachment() — shared metadata layer for Finance receipts (EXPENSE) and HR documents (EMPLOYEE), same encrypt/shard/pin client pipeline as org_documents
org-activity-log.js	logOrgActivity()/listOrgActivityForRecord() — one domain-generic audit log every Business Operations/Finance/HR workflow writes into
rag/collections.js, chunking.js, embeddings.js, sanitize.js, vectorStore.js, retrieve.js, ingest.js, youtubeTranscript.js, queryCache.js, metrics.js	RAG pipeline — see Section 11 for function-level detail
rag/sources/{docsSources,securitySources,learnSources}.js	Per-domain source adapters feeding ingest.js — DOCS_SOURCES, SECURITY_SOURCES, LEARN_STATIC_SOURCES + ensureVideoTranscriptIngested()
dataroom.js	Investor Data Room visitors/sessions/documents/views
activity.js	DAU/WAU ping recording + stats
watcherPioneer.js, watcherSocial.js, watcherTelegram.js	Watcher Pioneer enrollment/qualification; Google and Telegram sign-in (added October 2026)
feedback.js	Bug reports / idea submissions
metadata-auth.js	Shared signed-message verification helper (verifyMetadataAuth)
email.js	Resend-backed transactional email (magic links, Data Room verify)

SECTION 06

API Route Reference (by domain)

DOMAIN	ROUTES
Security (public)	GET /api/security/{threat,feed,stats,policy}, POST /api/security/report, GET /api/security/reputation/[address], POST /api/security/events, GET /api/security/events
Security (admin)	GET /api/admin/security/{threats,nodes}, POST /api/admin/security/threats/[id]/override, GET|POST /api/admin/security/policy
Security (AI)	POST /api/ai/security-chat
Security (cron)	GET /api/security/cron/checkpoint-reputation (CRON_SECRET-gated)
Learn	GET /api/learn/config, GET /api/learn/search, GET /api/learn/video/[videoId], GET|POST /api/learn/saved, DELETE /api/learn/saved/[videoId], GET|POST /api/learn/progress, POST /api/learn/report, POST /api/learn/analytics
Learn (AI)	POST /api/ai/learn-chat
Business Workspace (auth)	POST /api/orgs/create, /api/orgs/login/google, magic-link login/verify
Business Workspace (org)	department/project CRUD, document upload/list/retrieve/transition, permissions grant/update/revoke, share create/list/revoke, GET /api/orgs/share/[token] (public resolve)
Business Workspace (billing)	GET /api/orgs/billing, POST /api/orgs/billing/checkout, POST /api/orgs/billing/portal, GET /api/orgs/billing/plans
Business Workspace (AI)	POST /api/ai/business-chat
Business Workspace (tasks/CRM/procurement/inventory)	/api/orgs/tasks/**, /api/orgs/crm/{contacts,deals}/**, /api/orgs/procurement/{suppliers,requests,orders}/**, /api/orgs/inventory/{products,warehouses,movements}/**, GET /api/orgs/dashboard (aggregate summaries per module)
Business Workspace (finance)	/api/orgs/finance/invoices/** (incl. [id]/transition, [id]/activity), /api/orgs/finance/expenses/** (incl. [id]/attachments, [id]/transition), /api/orgs/finance/payments/** (incl. [id]/approve), GET /api/orgs/finance/reports (?type=revenue|expenses|outstanding|paid-unpaid&format=json|csv)
Business Workspace (HR)	/api/orgs/hr/employees/** (incl. [id]/attachments, [id]/leave-balance, [id]/transition, [id]/activity), /api/orgs/hr/leave-requests/** (incl. [id]/transition), POST|DELETE /api/orgs/hr/departments/[id]/manager
Business Workspace (finance/HR cron)	GET /api/cron/invoices-mark-overdue (CRON_SECRET-gated, nightly SENT→OVERDUE flip)
RAG (public, via existing assistants)	Consumed inside POST /api/ai/chat (Docs), /api/ai/security-chat, /api/ai/learn-chat — no separate public RAG endpoint
RAG (admin)	GET /api/admin/rag/stats, POST /api/admin/rag/reingest
RAG (cron)	GET /api/cron/rag-reingest (CRON_SECRET-gated)
Data Room (public)	POST /api/dataroom/request-access, GET /api/dataroom/verify, GET /api/dataroom/session, POST /api/dataroom/accept-nda, GET /api/dataroom/documents, GET /api/dataroom/documents/[id]/stream, POST /api/dataroom/documents/[id]/view-event
Data Room (admin)	GET|POST /api/admin/dataroom/documents, DELETE /api/admin/dataroom/documents/[id], GET /api/admin/dataroom/visitors, POST /api/admin/dataroom/visitors/[id]/revoke
Nodes	POST /api/nodes/register, POST /api/nodes/heartbeat, GET /api/nodes/settlements/release (cron)
Referrals	POST /api/referrals/{activate,initiate,webhook}, GET /api/referrals/{status,history,leaderboard}
Watcher Pioneer	POST /api/watcher/{enroll,qualify,link} and /api/watcher/telegram/{start,webhook}; GET /api/watcher/status and /api/watcher/telegram/poll
Activity	POST /api/activity/ping, GET /api/admin/activity

DOMAIN	ROUTES
Other	POST /api/faucet, POST /api/stripe-webhook, POST /api/create-payg-checkout-session, POST /api/feedback/submit, POST /api/feedback/upload, GET /api/admin/feedback

SECTION 07

MongoDB Collections Reference

COLLECTION GROUP	KEY COLLECTIONS
Security	security_reports, security_threats, security_policy, security_events, security_reputation_cache
Learn	learn_saved, learn_progress, learn_search_cache (TTL 24h), learn_video_cache (TTL 24h), learn_reports, learn_analytics_events
Business Workspace (core)	orgs, org_members, sessions, magic_links, departments, projects, documents, document_permissions, document_shares, document_activity, project_members
Business Operations	tasks, crm_contacts, crm_deals, suppliers, purchase_requests, purchase_orders, warehouses, products, stock_levels, stock_movements, org_activity (one shared domain-generic audit log for every module below, not just Business Operations)
Finance & HR	invoices, expenses, payments, employees, leave_requests, attachments (shared metadata store — EXPENSE and EMPLOYEE relatedRecordType)
RAG	rag_chunks (with Atlas Vector Search + Atlas Search indexes), rag_sources, rag_ingestion_runs, rag_query_log, rag_embedding_cache
Data Room	dataroom_documents, dataroom_visitors, dataroom_magic_links, dataroom_sessions, dataroom_views
Activity	activity_pings — one doc per {surface, identityId, date}, unique compound index
Referrals / Watcher Pioneer / Feedback	referrals, watcher_pioneers / watcher_sessions / watcher_program_counters / watcher_compensation_log / watcher_identities / watcher_wallet_links / watcher_telegram_logins, feedback_submissions (each with their own lib.js as listed in Section 05)

SECTION 08

AI Assistant Tool-Calling Pattern

Identical shape across all four assistants (src/app/api/ai/{chat,business-chat,security-chat,learn-chat}/route.js) — chat/route.js is the Docs Assistant, the other three are named for their domain:

- Model: gemini-flash-latest, thinkingConfig: { thinkingLevel: "low" }, maxOutputTokens: 800
- MAX_TOOL_ROUNDS = 5 — a bounded loop of generateContent → check functionCalls → run tools → feed functionResponse back in
- generateContentWithRetry() — retries on HTTP 429/503 with a [700ms, 1800ms] backoff schedule
- Route-level: maxDuration = 90, not streaming (deliberate — unpredictable number of tool round-trips before a final answer)
- Per-assistant: buildXContext() computed once per request, X_TOOL_DECLARATIONS (Gemini Type.OBJECT schemas), runXTool(name, args, ctx) dispatcher, xSystemInstruction() builder

SECTION 09

Auth Patterns — Technical Detail

PATTERN	IMPLEMENTATION DETAIL
Wallet signed-message	ethers.verifyMessage(reconstructedMessage, signature) === expectedAddress, 5-minute timestamp freshness window, reimplemented per-feature (metadata-auth.js has the canonical version, security.js/watcherPioneer.js reimplement locally by convention)
Magic-link	generateToken() (random), hashToken() (stored hashed, never plaintext), TTL-indexed Mongo collection, consumeLoginToken() atomic findOneAndUpdate to prevent replay
Session (web)	HttpOnly cookie, createSession(email) helper in orgs.js
Session (mobile)	Bearer token, same session concept, stored via expo-secure-store
Share tokens	generateShareToken()/hashShareToken(), atomic findOneAndUpdate-based single-consume pattern (document-permissions.js), reused as a template for Data Room's magic-link tokens

SECTION 10

Testing & Verification Conventions

- Hardhat tests: test/security-layer.test.js (17 tests — access control, confirm/re-confirm, false-positive override, 4-signer simulation)
- Node backend tests: test/*.test.mjs run via node --test against a real MongoDB instance (not mocked) — security.test.mjs (11), learn.test.mjs, dataroom.test.mjs, activity.test.mjs, feedback.test.mjs, watcher-pioneer.test.mjs — 140/140 passing as of the Security Layer build, no regressions
- Business Operations: business-operations-tasks.test.mjs, crm-workflow.test.mjs, procurement-workflow.test.mjs (incl. real inventory-ledger integration — a receipt genuinely moves stock), document-permissions.test.mjs
- Finance & HR: finance-workflow.test.mjs (8 tests — invoice lifecycle, cron overdue-flip + idempotency, org isolation, expense approval gate, payment record→approve replay-safety), hr-workflow.test.mjs (9 tests — employee lifecycle, leave approval + computed-balance correctness, self-access vs. HR-role vs. Department Manager scope) — both caught real bugs before shipping: a race condition in PO receiving (stock movements were applying before the PO's own optimistic-concurrency-guarded update), and a Department Manager visibility bug in getAccessibleScope() (Section 12)
- RAG: rag-ingestion.test.mjs, rag-security.test.mjs, rag-attribution.test.mjs, rag-learn-transcript.test.mjs, rag-retrieval.test.mjs (28 tests total) — run against the real Atlas cluster and real Gemini embeddings, not mocked; the relevance threshold (Section 11) was empirically calibrated from live measurements taken during this test work, not guessed
- On-chain simulation: scripts/simulate-security-nodes.js — 4 throwaway signers report a clearly-fake test indicator end-to-end through to a real on-chain confirmThreat() transaction on BSC Testnet
- Manual browser verification is the norm for UI work in this codebase — curl/fetch checks for API correctness, then a real browser pass (or explicit statement that one hasn't happened yet) before anything is called "verified"

SECTION 11

RAG Pipeline — Function Reference

FUNCTION	FILE	PURPOSE
chunkMarkdownByHeading() / chunkParagraphs() / chunkStructuredSections() / chunkQaPairs()	rag/chunking.js	Source-shape-specific chunkers — structured-sections for fundraising-docs' {sections: [{blocks}]} shape, qa-pairs for FAQ arrays, heading-split for markdown, paragraph-split as the fallback
embedText() / embedChunkText() / embedQueryText()	rag/embeddings.js	Wraps Gemini's models.embedContent (gemini-embedding-001, 768 dims) — RETRIEVAL_DOCUMENT taskType for ingestion, RETRIEVAL_QUERY for search; content-hash-cached in rag_embedding_cache
sanitizeChunkText() / wrapContextBlock()	rag/sanitize.js	Injection-pattern neutralization before a chunk is ever fed into a model prompt, and delimited wrapping so retrieved context can't be confused with the system prompt
ensureIndexes() / upsertChunks() / deleteBySource() / deleteObsoleteChunks() / hybridSearch()	rag/vectorStore.js	Idempotent index creation ($vectorSearch + $search via collection.createSearchIndex()); hybridSearch() runs both queries and merges via Reciprocal Rank Fusion in application code
retrieveContext() / formatAttribution()	rag/retrieve.js	The one function every assistant calls. DEFAULT_MIN_RELEVANCE = 0.8 — hasResults gates purely on top vector score, no keyword-match override (removed after it let one irrelevant query pass)
ingestSource() / ingestAllStaticSources() / deleteSource()	rag/ingest.js	Content-hash diffing per source: insert new chunks, update changed ones, skip unchanged ones, remove chunks whose source content no longer produces them
fetchYouTubeTranscript(videoId)	rag/youtubeTranscript.js	Scrapes the public timedtext caption-track URL out of a video's watch-page HTML — unofficial (the official Data API's captions.download needs OAuth as the channel owner), always fails gracefully to null, never throws
recordRetrieval() / recordIngestionRun()	rag/metrics.js	Writes to rag_query_log / rag_ingestion_runs — fail-open, a metrics write never blocks the actual retrieval/ingestion it's recording

> DOCS_SOURCES (rag/sources/docsSources.js) covers inaya-knowledge.js, business-workspace-guide.md, the FAQ page's faqs export, and all 15 scripts/fundraising-docs/content/*.js files — explicitly excluding custody-sdk's own docs (a nested, separately-git-excluded repo — see Section 20's build-breakage history for why touching it at all is risky) and the legacy KNOWLEDGE_ARTICLES collection.

SECTION 12

Business Operations & Finance/HR — Workflow Reference

Every transition below follows the same shape: one findOneAndUpdate({..., status: fromState}, {$set: {status: toState}}) — a mismatched current status is a clean 409, no extra locking needed.

MODULE	TRANSITIONS
task-workflow.js	submit: DRAFT→IN_PROGRESS, complete: IN_PROGRESS→DONE, reopen: DONE→IN_PROGRESS, cancel: *→CANCELLED
deal-workflow.js	stage transitions across the pipeline, terminal: →WON / →LOST
purchase-request-workflow.js	submit: DRAFT→PENDING_APPROVAL, approve/reject: PENDING_APPROVAL→APPROVED|REJECTED (requiresManage), cancel: DRAFT|PENDING_APPROVAL→CANCELLED
purchase-order-workflow.js	submit→approve→order→receive (partial and full) — receivePurchaseOrder() applies inventory stock movements only AFTER the PO's own optimistic-concurrency-guarded status update succeeds (reordered after a real race condition was caught — see Section 10)
invoice-workflow.js	send: DRAFT→SENT, markPaid: SENT|OVERDUE→PAID, cancel: DRAFT|SENT|OVERDUE→CANCELLED, markOverdueInvoices() (cron-only): SENT→OVERDUE
expense-workflow.js	submit: DRAFT→PENDING_APPROVAL, approve/reject: PENDING_APPROVAL→APPROVED|REJECTED (requiresManage: canManageFinance), cancel: DRAFT|PENDING_APPROVAL→CANCELLED
employee-workflow.js	activate: ONBOARDING→ACTIVE, placeOnLeave: ACTIVE→ON_LEAVE, returnFromLeave: ON_LEAVE→ACTIVE, terminate: ACTIVE|ON_LEAVE→TERMINATED (requiresManage: canManageHR)
leave-workflow.js	approve/reject: PENDING→APPROVED|REJECTED (requiresManage: canManageHR), cancel: PENDING→CANCELLED (requester or canManageHR); getLeaveBalance() = annualLeaveAllocationDays − sum(this year's APPROVED day-spans), computed on every call

getAccessibleScope() extension pattern. Each module adds its own visibleX array (visibleTasks, visibleContacts, visibleInvoices, visibleEmployees, ...) to the same resolver rather than each having a separate scope function. Finance's arrays are gated by canAccessFinance on top of department scope; HR's visibleEmployees unions HR-scoped employees, Department-Manager-scoped employees (managedDepartmentIds — NOT filtered against the caller's own already-visible departments, a real bug fixed during this layer's testing, see the founder architecture doc's Section 24), and the caller's own self-record, deduplicated by _id.

SECTION 13

Multi-Chain Bridge & Interop Reference (September 2026)

FUNCTION / CONTRACT	PURPOSE
InayaTokenBridgeHome.bridgeOut(...)	BSC-side lock/burn entry point, source of every cross-chain transfer
InayaTokenBridgeSpoke.executeMessage(...) / receive_message(...)	Remote-chain entry point (EVM name / Solana-Move name) — validates threshold validator signatures, mints/releases on the destination
InayaChainRegistry.setSpokeBridgeAddress() / registerRemoteChain() / setTrustedRemoteContract()	Owner-only wiring — a chain has no real route until all three are called for it, not just deployed
listChainCapabilities() / getChainCapability(chainId)	src/lib/chain-adapters/registry.js — the single source a UI reads before ever describing a chain as 'supported'; SUPPORT_LEVELS run DISCOVERED(0) → READ_ONLY → WALLET → MESSAGE → TOKEN_TRANSFER → STAKING → FULL_ECOSYSTEM(6), each backfilled only from what's actually been verified
WormholeProvider.js	Wraps Wormhole's WTT/lock-and-mint flow; chain-id mapping is the thing that broke first (Sepolia/testnet IDs differ from their mainnet counterparts unlike BSC/Avalanche)
GET/POST /api/interop/wtt/{initiate,relay,status}	Relay cron (*/5 min) fetches the Guardian-signed VAA via a plain REST call to Wormholescan's public API (not the @wormhole-foundation/sdk package — that broke the Next.js build, see Section 26 of the architecture doc) and completes the transfer via Inaya's relayer wallet

Per-chain proof records (real tx hashes both sides): deployments/bridge/{avalancheFuji,solanaDevnet,hederaTestnet,aptosTestnet,suiTestnet}.json; deployments/interop/wormhole-wtt/bscTestnet-attestation.json.

SECTION 14

Storage Backup & Redundancy Reference (September 2026)

FUNCTION / CONTRACT	PURPOSE
InayaBackupRegistry.sol	0x062c341aE4f11CB1dEa1B0D3930d52902F97f48a (BSC Testnet) — records redundancy commitment + health-state transitions only, never replica data
InayaKernel.Backup.getBackupStatus / getBackupHealth / getRedundancyStatus / getRecoveryStatus / requestRecovery	custody-sdk client, published to npm
Health states	PROTECTED / REBUILDING / DEGRADED / RECOVERY_REQUIRED / RECOVERY_FAILED — pure state machine, 19/19 unit tests
Pin-status check	Every 15 min, 3-strike grace window before flagging a replica
Integrity check	Daily, content-hash comparison, zero grace — catches real corruption immediately

Providers: Pinata + Filebase (S3-compatible, IPFS-backed, separate infrastructure/failure domain). Filebase gotcha: its S3 endpoint is s3.filebase.io, not .com; the AWS SDK's default auto-attached request checksum causes a generic AccessDenied against it unless disabled. Full writeup: docs/backup-redundancy-architecture.md.

SECTION 15

AI Guarded Execution & Audit Trail Reference (September 2026)

FUNCTION	PURPOSE
proposeAiAction({orgId, assistantSurface, toolName, targetRecordType, targetRecordId, proposedAction, args, actorEmail, canPropose})	Inserts a PENDING_APPROVAL request; idempotencyKey = sha256(orgId+toolName+targetRecordId+args+hourBucket) dedupes same-hour repeats
reviewAiAction(..., canApprove)	Approve sets unlockAt = now + 36h (SETTLEMENT_DELAY_MS); reject is terminal
classifyRisk(targetRecordType, proposedAction)	Never returns undefined — unmatched combinations default MEDIUM; EXPENSE/INVOICE/PURCHASE_ORDER default HIGH
api/cron/execute-approved-ai-actions	The only code path that calls a real transitionX() for an AI-originated change, and only once unlockAt has passed; runs as a synthetic org-manager membership, attributed in the log to the human approver
appendAuditEntry({orgId, recordType, recordId, actorEmail, action, previousState, newState, metadata})	src/lib/auditChain.js — entryHash = sha256(prevHash + canonicalJSON(eventFields)); optimistic CAS on audit_chain_heads, retries on conflict, never corrupts under contention
verifyChainIntegrity(orgId)	Walks the full chain, recomputes every hash; returns {valid:false, brokenAtSeq, reason} on the first mismatch (gap/reorder, prevHash mismatch, or content altered after write)
GET /api/orgs/audit/export?orgId=&format=json|csv	Org-scoped (owner/admin only) full-chain export — every field needed to independently recompute and verify, so a customer never has to trust a UI badge

9 gated domains (EXECUTORS map in ai-action-requests.js): TASK, EXPENSE, DOCUMENT, EMPLOYEE, INVOICE, LEAVE_REQUEST, PURCHASE_ORDER, PURCHASE_REQUEST, DEAL — one corresponding propose_* tool each in ai-business-tools.js. State machine: PENDING_APPROVAL → APPROVED → QUEUED → EXECUTED|EXPIRED, or PENDING_APPROVAL → REJECTED, or APPROVED → CANCELLED (only before unlockAt). 19 automated tests, 11 adversarial.

SECTION 16

Web3 App Store, NFT Vault & MFA Reference (September 2026)

FUNCTION / ROUTE	PURPOSE
submitAppListing / listApprovedListings / listPendingListings / listListingsBySubmitter / reviewAppListing	src/lib/appStoreListings.js — full submission→threat-check→review→approve pipeline
POST /api/apps/submit, GET /api/apps/listings, GET /api/apps/my-listings	Developer-facing App Store API
GET /api/admin/app-store/pending, POST /api/admin/app-store/[slug]/review	Admin review queue, passphrase-gated same as /admin/audit
src/app/apps/embed/[slug]/page.js	Sandboxed iframe host for externally-hosted app submissions — no allow-same-origin + allow-scripts together
checkNftContractSupport / discoverOwnedTokens	src/lib/nftDiscovery.js — ERC-721 + Enumerable only, no indexer
signNftBackup	src/lib/nftBackupAuth.js — wallet-signature auth reused for NFT backup requests
POST /api/nft/backup, GET /api/nft/backups	NFT Vault backup API
src/components/AddressRiskCheck.js	Debounced live Security Layer threat-registry check, wired into the Bridge page's recipient field
MfaSettings.js / MfaVerifyScreen.js	TOTP + Firebase-Phone-Auth SMS MFA for Business Workspace login

SECTION 17

Sovereign Enterprise OS Reference (September 2026)

MODULE	KEY EXPORTS / FUNCTIONS
src/contexts/OrgContext.js	OrgProvider, useOrg() → {email, membership, orgId, role, orgs, switchOrg, logout, can: {manageOrg, accessDepartment, manageFinance, accessFinance, manageHR, accessHR, isDepartmentManager}}
src/contexts/WalletContext.js	WalletProvider, useWallet() → {walletAddress, walletBalance, isConnected, selectedWalletName, signMessage}
src/lib/orgGates.js	Client-safe extraction of orgs.js's 7 pure permission gates — re-exported from orgs.js under the same names for the ~60 existing server-side importers
src/lib/trustHealth.js	computeTrustHealthSnapshot({scope:"org"|"wallet", ...}) → {overallStatus, auditTrail|security, aiActions|backup, businessHealth, scopeNotes}
GET /api/orgs/trust-health, GET /api/wallet/trust-health	Org: requireMembership-gated. Wallet: unauthenticated, aggregate counts only
src/lib/notifications.js	createNotification (idempotent on dedupeKey), listNotificationsFor, markRead, markAllRead — collections: notifications, notification_reads
GET/POST /api/orgs/notifications/*, /api/wallet/notifications/*	Org: session-cookie auth. Wallet: GET unauthenticated, mutations signed via verifyMetadataAuth's existing message format
src/lib/orgSearch.js, src/lib/walletSearch.js	searchOrg({orgId, membership, email, query}) over getAccessibleScope(); searchWallet({walletAddress, query}) over metadata_files
src/components/CommandPalette.js	Cmd/Ctrl+K overlay, surface-agnostic — onSelect(result) left to the caller's own real navigation function
src/lib/activityCenter.js	generateWhatChanged({scope, ..., period}) → {sections: [{module, bullets}]}; reuses business-brief.js's BRIEF_PERIODS
src/lib/ai-os-router.js	buildOsContext, getOsToolDeclarations(scope), runOsTool(name, args, ctx), osSystemInstruction — business_/security_ prefixed dispatch
POST /api/ai/os-chat, POST /api/ai/os-chat-wallet	Org: requireMembership-gated. Wallet: identityId passed directly, matching security-chat's existing precedent
src/components/business/OsHomeView.js, src/components/OsHomeSection.js	The composed OS Home screen per surface — TrustHealthCard, OS Assistant widget, What-Changed preview, module tiles, surfaced links
open_module_window(label, path)	Tauri command, both inaya-desktop and inaya-dapp-desktop's src-tauri/src/lib.rs — pops a module into its own native window

Every org-scoped route follows the exact ensureOrgIndexes()+requireMembership()+getAccessibleScope()-or-narrower pattern dashboard/route.js already established; every wallet-scoped GET follows the same unauthenticated-aggregate-only trust tier as the pre-existing list-files/backup-status routes. No existing route, collection, or component was renamed, removed, or had its behavior changed — additive only.

SECTION 18

Healthcare & Legal OS Reference (September 2026)

FUNCTION / ROUTE	PURPOSE
health-patients.js: createPatient, assignCareTeamMember, findDuplicatePatientCandidates, mergePatients	Patient registry; care-team assignment is the join row getAccessibleScope() reads for visibility
health-clinical-workflow.js: TRANSITIONS (draft→review→sign→lock), amendClinicalRecord	Locked records are never overwritten — amend creates a new linked record, marks the original AMENDED
health-consent-workflow.js, health-roi-workflow.js, health-breakglass.js	Consent record/withdraw; ROI request→authorize→review (hands off to export-center.js); break-glass = a real time-limited care-team assignment, not a bypass path
legal-matter-workflow.js: createMatter, assignMatterTeamMember; legal-conflict-workflow.js: searchConflicts, recordConflictCheck	Matter lifecycle + team assignment; conflict status is always potential/cleared/escalated, never a raw boolean
legal-evidence.js, legal-custody.js: recordCustodyEvent; legal-hold-workflow.js: createLegalHold, releaseLegalHold	Evidence metadata over an org_documents FK; every custody event chained via appendAuditEntry; holds gate retention.js's checkDispositionAllowed()
trust-accounting.js: recordDeposit, recordWithdrawal, getMatterTrustBalance	Atomic balance recheck before every withdrawal — no jurisdictional (IOLTA) compliance claimed
ai-health-tools.js, ai-legal-tools.js	buildXContext/X_TOOL_DECLARATIONS/runXTool/xSystemInstruction — read/summarize/draft only, wired into ai-os-router.js with health_/legal_ prefixes
industry-config.js: requireVertical(orgId, expectedVertical)	The door-lock — every Health/Legal route calls this right after requireMembership(); 403 on any mismatch, fail-closed as 'general' with no vertical set
25 routes under /api/orgs/health/*, /api/orgs/legal/*	Patients, consents, ROI, break-glass, appointments, billing, care-team, research (health); clients, prospects, matters, matter-team, conflict-checks, evidence, holds, discovery, redaction, deadlines, contracts, entities, trust-accounting, time-entries, billing (legal)
HealthView.js, LegalView.js (src/components/business/)	The only two modules with an actual screen today — registry/list + create + detail modal, mirroring CRMView.js's structure
src/lib/integrations/{health,legal}/*Adapter.js + adapterStub.js	13 documented third-party interfaces (FHIR, HL7, DICOM, lab, pharmacy, claims, e-filing, legal research, e-signature, email, calendar, accounting, SSO) — configured:false until real env credentials are present, never fabricated

Patient/matter visibility is assignment-based (health_care_team_assignments/legal_matter_team_assignments), a genuinely different model from every other Business Workspace module's department-scoping — wired directly into getAccessibleScope() rather than a parallel resolver. 20 domain modules have real backend + API; only 2 (Patients, Matters) have a screen. Two real bugs were caught and fixed during this build (break-glass had no expiry enforcement; 14 POST route handlers referenced an out-of-scope variable after the vertical-lock was added) — both now covered by permanent regression tests.

SECTION 19

Financial Services & Regulated Enterprise OS Reference (2026)

FUNCTION / ROUTE	PURPOSE
compliance-controls.js, compliance-evidence.js, control-testing.js	Control library + evidence vault; a failed control test auto-opens a Finding via the shared finding lifecycle
compliance-policies.js: createPolicyDraft→submitForReview→approve→publishPolicy, amendPolicy	Publish sets immutable:true; amendPolicy is the ONLY path forward from PUBLISHED, always inserting version+1, never mutating the published row
regulatory-examination-access.js: createExaminerMagicLink, exchangeMagicLink	Scoped {examinationId, requestIds} access, deliberately its own token pattern (not a retrofit of the unrelated investor dataroom.js)
financial-entities.js, financial-funds.js: isFundTeamMember	Entity-scoped fund visibility — same assignment-based shape as isCareTeamMember/isMatterTeamMember
investment-committee.js, cap-table.js, board-management.js	IC decision lifecycle (finalized decisions are versioned, never overwritten); cap-table is an ingest/evidence layer, never a transactional share registry
privileged-access.js: requestElevation→approveElevation, grantBreakGlass, reviewSession	Cross-vertical from day one — the approver must be a different person than the requester; break-glass is ACTIVE immediately but always forces a post-event review
integrations.js: INTEGRATION_PROVIDERS, configureIntegration, recordSyncRun	A connection can only ever reach ACTIVE through a real recorded sync run — never through configuration alone
ai-audit-tools.js, ai-regulatory-tools.js + 7 more role-specific copilots	All 100% read/summarize/flag — a compliance-certification request is refused at the tool layer before any summarization runs
~20 API subtrees under /api/orgs/{regulated,financial,private-capital}/*	Every route: ensureOrgIndexes → requireMembership → requireVertical → lib call, statically verified by vertical-lock-wiring.test.mjs
RegulatedView.js, FinancialView.js (src/components/business/)	Full tab-router UI — Controls, Evidence, Findings, Policies, Risk, Audits, Examinations, Dashboard (Regulated); equivalent depth for Financial

All ten SOW phases are live-verified end-to-end on web: control creation/testing/finding lifecycle, the policy publish-immutability guard, an examiner magic-link's one-time-use cycle, entity-scoped fund/deal visibility, and the board-report/export-package immutability guards. Nothing on inaya-mobile for any of these three verticals yet. Every third-party integration (fund admin, custodian, KYC/AML, etc.) is a documented, honest not-configured stub.

SECTION 20

Government & Public Sector Sovereign OS Reference (2026)

FUNCTION / ROUTE	PURPOSE
citizen-records.js: createCitizenRecord, assignCitizenRecord, requireCitizenRecordAccess	requireCitizenRecordAccess() is the load-bearing gate — department membership or governmentRole:'staff' alone is never enough, only an explicit assignment (or org owner/admin) grants access to one specific record
government-cases.js: CASE_STATES (OPEN→ASSIGNED→IN_PROGRESS→PENDING_REVIEW→RESOLVED→CLOSED)	Same TRANSITIONS-map + atomic findOneAndUpdate template as incidents.js; a case linked to a citizen record inherits that record's need-to-know check
policy-knowledge-base.js: createEntryDraft→publishEntry, amendEntry	compliance-policies.js's exact publish-immutable lifecycle, recontextualized — no updateEntryDraft() path reachable once PUBLISHED
government-audit.js: logCitizenRecordAccess, logGovernmentDocumentRead	Government-vertical-only: every document READ is chain-logged, not just writes — an additive branch inside the existing retrieve route, gated so no other vertical's behavior changes
government-dashboard.js: getGovernmentDashboard	Operations + security readiness as two separate honest panels — a fresh org with zero audit-chain entries reports 'unknown,' never a fabricated 'valid'
ai-government-tools.js: search_citizen_records, summarize_case, get_dashboard_summary	100% read-only; need-to-know is re-checked INSIDE the tool per record, not just trusted from the API layer above it
privileged-access.js (reused, zero new code)	Break-glass access for Government orgs — already cross-vertical since §33, no government-specific fork needed
integrations.js: 5 new government_systems providers	Civil/national ID registry, legacy government ERP, GIS/land records, public records portal, interagency data exchange — all configured:false stubs
11 routes under /api/orgs/government/*	citizen-records (+assign), cases (+transition), policy-kb (+transition/publish/amend), dashboard — statically verified by the extended vertical-lock-wiring.test.mjs
GovernmentView.js (src/components/business/)	Tab-router UI — Citizen Records, Cases, Policy Knowledge Base, Dashboard — wired into the Business Workspace nav and org-creation flow, confirmed rendering live

51 new automated tests, including the two load-bearing properties written before their implementation: citizen-record access requires an actual assignment, and a published policy knowledge base entry can never be mutated in place. Nothing on inaya-mobile yet. No government certification, accreditation, FedRAMP authorization, or jurisdiction-specific compliance claim exists or is claimed anywhere.

SECTION 21

Institutional Trust Infrastructure Reference (September 2026)

FUNCTION / ROUTE	PURPOSE
evidence.js: getEvidenceTrail, verifyOrgEvidenceIntegrity, exportEvidencePackage	Read-only merge of listOrgActivityForRecord() + linked aiActionRequests into one timeline; integrity check is a pass-through to auditChain.js's verifyChainIntegrity()
org-trust.js: proposeTrustRelationship, respond, revokeTrustRelationship, isTrustedAccess	New orgTrustRelationships collection; accept/reject filtered by findOneAndUpdate({toOrgId: caller}) so an org can never approve its own proposal; isTrustedAccess is a query primitive no other module calls yet
POST/GET /api/orgs/trust-relationships, PATCH /api/orgs/trust-relationships/[relationshipId]	Session-authenticated cross-org trust CRUD
api-keys.js: generateToken/hashToken convention, requireApiKey	Raw key shown once at creation, only hash persisted; resolves to a synthetic owner-level membership permanently bound to the key's own orgId — no request parameter can override it
POST/GET /api/orgs/api-keys, DELETE .../[apiKeyId]	Session-authenticated key management, owner/admin only
GET /api/public/v1/audit/verify, /api/public/v1/evidence, /api/public/v1/permissions/check	API-key-authenticated (Bearer). evidence requires both recordType+recordId (no org-wide dump); permissions/check reports an org-level capability flag, not per-user RBAC
institutional-attention.js: getAttentionItems	AI-tool-only (get_attention_items in ai-os-router.js), not a REST route; merges pending-approvals (re-checked against the real approval gate), overdue tasks, pending documents
src/app/trust/page.js — /trust	Public, unauthenticated. Client-side verifier reimplements auditChain.js's hash formula via Web Crypto; auto-detects genesis vs. scoped exports (fixed in 9345df5) and reports the correct guarantee strength for each
custody-sdk: TrustPlatform. {verifyAudit,getEvidence,checkPermission}	First-party SDK client for the three /api/public/v1 routes above (audit/verify, evidence, permissions/check) — a thin wrapper, not a second implementation

> Zero mainnet references across every file/route touched by this section, confirmed by direct grep. Every module ships a co-located test file (org-trust, evidence, institutional-attention, api-keys), none mocked against a fake database.

SECTION 22

Autonomous Resilience Layer Reference (September 2026)

FUNCTION / ROUTE	PURPOSE
resilience-policy.js: createPolicy, updatePolicy, evaluateRtoRpo	requiredRTOMinutes/RPOMinutes + critical-asset categories + test frequency; evaluateRtoRpo is a plain actual<=required comparison, no fuzzy scoring
resilience-canary.js: ensureResilienceHome, createCanaryAsset	Synthetic org-owned documents (isResilienceCanary:true, hidden from normal browsing) pushed through the real InayaKernel.disperseAndSlice() + real pinning-provider .pin() + real backupEngine.replicateShard() — never a mock of these functions
resilience-orchestrator.js: runResilienceTest, runScheduledResilienceTests	13-step test: real reconstruction via InayaKernel.reconstructAndDecrypt(), real getPinStatus() dependency check, real getDocumentAccessLevel() permission check, wall-clock RTO, replica-staleness-based RPO; atomic RUNNING-status duplicate-run guard
GET /api/cron/resilience-tests	CRON_SECRET Bearer-gated daily sweep, registered in vercel.json; per-policy try/catch so one broken policy never blocks the rest
resilience-status.js: deriveState	Pure function → Unknown / Test Due / Verified / Failed / Degraded, unit-tested directly; a PAUSED policy always reports Unknown regardless of test history
GET /api/orgs/resilience/status, POST /api/orgs/resilience/policies/[policyId]/run	Dashboard read + manual 'Run now' (canManageOrg-gated, triggeredBy:'manual' vs. cron's 'schedule')
ai-resilience-tools.js: get_resilience_status, get_last_test_result, list_failed_assets, explain_test_failure, trigger_resilience_test_now	First four are structurally read-only (no mutation code path exists); trigger_resilience_test_now can only start the same non-destructive test the cron runs, kept outside Guarded Execution because there is no destructive-workflow surface for it to gate

Two small, additive fixes to the pre-existing backupEngine.js's replicateShard() (expose providerRef; accept an explicit primaryProviderRef) were required and made in this same pass, both non-breaking. Verified live against real Filebase network calls during development.

SECTION 23

Node Operator Dashboard Reference (September 2026)

FUNCTION / ROUTE	PURPOSE
nodeOperatorAuth.js: loginWithWalletSignature, requireNodeSession, linkWalletToSession	New node_sessions collection, 12h cookie (inaya_node_session), reuses the existing verifyNodeAuth() signature check unchanged; fleet-linking requires the linked wallet's own signature
nodeUptimeHistory.js: getUptimeForWindow	Hourly-cron-populated node_uptime_snapshots; sets insufficientData:true and reports the real coverage window rather than backfilling or estimating history it doesn't have
nodeChainReads.js: getOnChainNodeInfo	Real ethers.JsonRpcProvider reads against the deployed node-registry contract (same address the existing settlement-release cron uses); tier/commission explicitly sourced on_chain since the off-chain Mongo tier field is set once and never revisited; fails honest-empty, never fabricates a tier
GET /api/nodes/operator/{me,uptime,qualification,rewards,events,network,fleet}, POST .../link, .../login, .../logout	network is the only unauthenticated route (public aggregate stats); every other route scopes strictly to the session's own wallet — no client-supplied node ID is ever trusted
GET /api/cron/nodes-snapshot	Bearer CRON_SECRET-gated hourly snapshot job, registered in vercel.json
src/app/operator/page.js + OperatorOverview/Uptime/Qualification/Rewards/Events/Network/Fleet.js	Seven-tab dashboard, standalone MetaMask connect-and-sign login independent of the dApp's WalletContext and Business Workspace's session

No claim/release control anywhere in the dashboard — settlement stays fully automated by the pre-existing timelock cron. Public /network reports off-chain tier distribution, not a live per-node chain read, an explicitly disclosed scalability trade-off. Tested with real ethers.Wallet signatures, treated internally as the highest-risk new surface in this pass.

SECTION 24

Four New Business Primitives Reference: Sign, DePIN Storage, Escrow, Attestation (September 2026)

FUNCTION / ROUTE	PURPOSE
signing-workflow.js: recordSignature, verifyInayaSignSignature, anchorSigningRequestOnChain, verifySigningRequest	Wallet path = real ethers.verifyMessage() ECDSA check, 5-min freshness window; session_consent path is explicitly labeled, never presented as equivalent; anchor is a real, optional transaction (batchRegisterAssets on the existing custody contract) over a completion-certificate hash, not a per-signature on-chain record
storage-manager.js: registerOrgStorageNode, getOrgStorageOverview, reportOrgStorageNodeHealth	New org-scoped control plane, confirmed unconnected to the public node-operator registry; eligibility is a static 'routing_not_yet_supported'; allocation/replica figures reused as-is from orgPlans.js + backupEngine.js, not recomputed
escrow-workflow.js: transitionEscrow, confirmMilestone, releaseMilestonePayment	State machine only flips a status field; the ONLY function moving money is releaseMilestonePayment, reachable only after a proposeAiAction() Guarded Execution approval plus the standard 36h SETTLEMENT_DELAY_MS; writes a real payments row, re-validates dispute status at execution time
financial-attestation.js: buildCanonicalDataset, generateAttestation, verifyAttestation	sha256 hash-commitment over real permission-scoped, currency-converted financial records; only revenue_threshold/expense_threshold implemented, solvency_threshold explicitly rejected (no assets/liabilities data model exists); verification independently re-runs the query and recomputes the commitment
POST/GET routes under /api/orgs/sign, /api/orgs/escrow, /api/orgs/storage, /api/orgs/attestations (~19 routes)	SignView, EscrowView, StorageManagerView, AttestationsView in src/components/business/ — real nav entries in page.js's Operations group, real org-scoped API-backed screens

Each module's own header comment states plainly what's real vs. not: Escrow is never described as on-chain or non-custodial anywhere in its code, routes, or UI; Attestation names the specific ZK libraries confirmed absent from the full dependency tree to avoid that label being misapplied; DePIN Storage Manager's region preferences are always labeled Declared, never Active. A real nested-object canonicalization bug found while testing Attestation was also fixed in Sign's anchor certificate, which shared the same flaw. Test coverage: inaya-sign, milestone-escrow, depin-storage-manager, financial-attestation.

SECTION 25

Business Workspace Operational Enhancements & Guided AI Assistant Reference (September 2026)

FUNCTION / ROUTE	PURPOSE
finance/invoices/[invoiceId]/pdf/route.js	Returns print-styled HTML, not a binary PDF — the route's own comment confirms no PDF library exists in the app; values pulled verbatim from the invoice record, never recalculated
currency.js: convert, SUPPORTED_CURRENCIES, USD_RATES	Static, explicitly dated reference-rate table (RATES_AS_OF), not a live FX feed by design; returns an error rather than guessing on unsupported pairs
inventory.js: transferStock, recordStockMovement	Two ledger movements + a materialized stock_levels update sharing one transferId; a real compensating ADJUSTMENT reversal if the second leg fails after the first commits
purchase-order-workflow.js: updatePurchaseOrderItems	Atomic findOneAndUpdate({status:'DRAFT'}) guard — editing outside DRAFT returns a real 409; reruns the same validator the create route uses
procurement/orders/[orderId]/email, procurement/reports	Real supplier email with real line items via the existing sendEmail() primitive (documented no-op if RESEND_API_KEY unset); 5 real report types incl. CSV export
insights route.js + business-insights.js: computeBusinessInsights	Real custom start/end date range on top of preset windows, validated (endDate<startDate → 400), computed via the same permission-scoped getAccessibleScope() every other route uses
business-chat/route.js — Gemini (gemini-3.5-flash-lite) via @google/genai, Groq fallback	Real, production-hardened LLM call: per-call timeout, request-level safety budget, retry-on-503 — added after an observed Gemini-queuing production issue
guided-workflow-catalog.js (static data, no LLM calls) + guided-tasks.js (state machine)	~10 hardcoded workflows; the model only ever relays pre-authored instruction text verbatim — structurally prevented from inventing step text; the task's own bookkeeping collection is the only thing it ever writes to
GuidedTaskPanel.js + inaya:guided-nav / inaya:guided-po-created custom events	Step advancement fires from real navigation events, real UI success events, or a manual 'I did this' confirmation — never the model's own assertion that a step is done

> Creating a new business record (contact, deal, PO, document) has no automated tool path at all — the assistant can only guide a human through the real UI. Any actual mutation it can propose still routes through the pre-existing Guarded Execution approval-plus-36-hour-delay pipeline described in §26/§31. No new mutation surface was added by the guided-task feature itself.

SECTION 26

Multi-Cloud Storage Compatibility & Enterprise Storage Governance Reference (September 2026)

FILE / ROUTE	PURPOSE
s3-compat/sigv4.js: verifySigV4Request, parseAuthorizationHeader	Real AWS SigV4 verification — canonical request → string-to-sign → derived signing key (HMAC chain) → signature; 15-min clock-skew window; UNSIGNED-PAYLOAD support
s3-compat/azureAuth.js: verifySharedKeyRequest, s3-compat/azureAuthMiddleware.js: authenticateAzureRequest	Azure Shared Key verification, SAS share links (azureSas.js, October 2026) and the real Microsoft Entra ID Bearer-token path (live Graph verification via integrationProviders/microsoft.js, mapped to org_members by email)
s3-compat/credentials.js: issueS3Credential, checkScope, normalizeScope	Credential issuance with optional scope {bucket, prefix, operations, expiresAt}; checkScope() is the single enforcement function, called centrally in auth.js/azureAuthMiddleware.js after signature verification
s3-compat/store.js, walletStore.js: putS3Object, deleteS3Object, assertNotProtected	Org- and wallet-side translation layers over org_documents/metadata_files; assertNotProtected() is the one chokepoint both the overwrite path and the delete path call through for Object Lock/Legal Hold enforcement
s3-compat/store.js: putBucketVersioning, listObjectVersions, restoreObjectVersion	Bucket-level versioningStatus (Unversioned→Enabled→Suspended, one-way); restore copies an old version's bytes forward as a new current version rather than resurrecting the old version id
s3-compat/store.js: putObjectRetention, putObjectLegalHold, putLifecyclePolicy, runLifecycleEnforcement	Object Lock (requires Versioning Enabled first; retention can extend, never shorten), Legal Hold, and per-bucket lifecycle rules with real enforcement that skips locked/held objects
api/s3/**/route.js, api/azure/**/route.js	Full S3 and Azure REST surfaces, including real sub-resources (?versioning, ?object-lock, ?versions, ?legal-hold, ?retention, ?versionId=, Put Block/Put Block List)
api/cron/s3-lifecycle/route.js	CRON_SECRET-gated route calling runLifecycleEnforcement() — same auth convention as api/cron/nodes-snapshot; registered in vercel.json's crons array
api/orgs/s3-compat/manage/route.js, S3CompatView.js	Session-authenticated Business Workspace management API + console — separate from the SigV4-signed protocol surface, so Business Workspace never needs to hold/sign with an S3 credential
backupEngine.js: replicateShard (now called from putS3Object)	Closes a real, previously-existing gap: S3/Azure-compat objects were pinned but never registered with the existing check-pins/verify-integrity/recovery crons
inaya-drive-helper/src/main.rs, sigv4.rs, s3client.rs	Standalone Rust crate (GPL-3.0, isolated on purpose) implementing WinFSP's FileSystemContext trait over a Rust port of the SigV4 signer, backed by the real /api/s3 endpoint
inaya-desktop/src-tauri/src/lib.rs: mount_inaya_drive, unmount_inaya_drive	Tauri commands that only spawn/kill inaya-drive-helper.exe as a child process — no GPL dependency in inaya-desktop's own Cargo.toml

Test coverage: test/s3-compat-capabilities.test.mjs (19 tests: scope enforcement, versioning, Object Lock, Legal Hold, lifecycle, health registration) plus the pre-existing test/s3-compat-sigv4.test.mjs (9) and test/s3-compat-store.test.mjs (9) — all passing, zero regressions. Full writeup: docs/storj-inspired-storage-capability-expansion-report.md and docs/multi-cloud-storage-compatibility-report.md.

SECTION 27

Storage Interoperability Expansion Reference — Empty Folders, Migration Tooling, Signed URLs & GCS Auth (September 2026)

FILE / ROUTE	PURPOSE
inaya-drive-core/src/lib.rs, s3client.rs, sigv4.rs	New standalone Rust crate extracted out of inaya-drive-helper: the SigV4-signed S3 client and signer, with zero Windows/WinFSP dependency, now linked unmodified by both inaya-drive-helper (Windows/WinFSP, GPL-3.0) and the new inaya-drive-helper-linux (Linux/FUSE via the MIT-licensed fuser crate, UNLICENSED) — one real client implementation, not a per-platform rewrite. Each platform's own filesystem-trait binding (WinFSP's vs. FUSE's) stays separate, since the two OS filesystem APIs aren't interchangeable
s3-compat/store.js: createS3Folder, getS3FolderInfo, deleteS3Folder, renameS3Folder	A real, durable folder record in a new s3_folders collection (mirroring the pre-existing Business Workspace metadata_folders shape) — a folder with zero objects is now listable via ?prefix&delimiter, where previously only shared object-key prefixes existed and an empty folder was invisible
inaya-migration-agent/bin/inaya-migrate.mjs	Standalone Node CLI: --source aws|azure|gcs, resumable via a local --manifest file (skips already-verified keys on rerun), --dry-run, --objects for a scoped subset; every source/destination credential stays local to the invoking machine, never transmitted to or stored by Inaya
s3-compat/signedUrl.js: createSignedUrl, verifySignedUrl	Query-string presigned GET URLs (X-Amz-Expires/X-Amz-Signature convention): reuses the existing SigV4 signing key derivation rather than a new secret, rejects on expiry, method mismatch, or any tampered query parameter
googleAuth.js: verifyGoogleIdToken, s3-compat/auth.js (Google ID-token branch)	Google Sign-In accepted as a direct auth path on the S3-compat endpoint: verifies the Google ID token's signature and audience server-side, then resolves it to an existing org_members row by verified email — no separate Google-only permission tier or credential type
middleware.js: bucket-from-hostname rewrite, gated by S3_COMPAT_VIRTUAL_HOST_BASE	Virtual-hosted-style addressing (bucket-name.base-host/key, matching real AWS/GCS convention) — the Host header is parsed and rewritten to the existing path-style route before any other S3/Azure code runs, so the protocol surface itself is unchanged; unset (off) by default
evidenceExporter.js: buildEvidencePackage, canonicalizeForExport	Reads an org's own existing audit chain and storage-protection settings (Object Lock, versioning, retention) into one canonicalized, hash-stamped package — no new data is generated or claimed, only exported
evidencePdf.js: renderEvidencePdf	Deterministic PDF rendering of the same package for a non-technical reader, hash printed on the document itself so a paper copy can still be checked against the JSON export

Test coverage: test/s3-compat-empty-folder.test.mjs (15 tests: placeholder-key creation/listing/deletion, cross-platform key-prefix handling, mount-restart persistence), test/signed-url.test.mjs (9: expiry, tamper rejection, method restriction, real wall-clock expiration — not a simulated clock), test/evidence-exporter.test.mjs (5), test/migration.test.mjs (6) — all passing against the real dev database, zero regressions in the pre-existing S3/Azure-compat suites.

Two corrected findings, kept honest rather than restated as always-true: Google's own "gcloud storage" CLI was earlier reported unable to reach a third-party endpoint like Inaya's — it can, through an official (Google-labeled "unstable") configuration option, verified with real uploads and downloads. The older "gsutil" tool still cannot, due to a genuine bug in Google's own software; a workaround narrowed it further but did not fully resolve it. Remaining disclosed limitation: macOS Drive support is written but not yet compiled or tested on real Mac hardware. Full writeup: docs/inaya-drive-empty-folder-creation-report.md, docs/enterprise-adoption-and-market-reach-report.md, and docs/gcs-compatibility-extension-report.md.

SECTION 28

AWS S3-Inspired Storage Feature Expansion Reference (September 2026)

FILE / ROUTE	PURPOSE
s3-compat/store.js, walletStore.js: putObjectTagging, getObjectTagging, deleteObjectTagging	Real S3 tag semantics enforced server-side (10-tag max, 128/256-char key/value limits), stored as an additive `tags` field on the existing object document, never a second metadata collection
s3-compat/store.js, walletStore.js: contentSha256 field on putS3Object	The real, unsalted SHA-256 of uploaded bytes, computed and stored alongside the pre-existing salted fileHash (salted for an unrelated legacy dedup reason and therefore not independently client-verifiable) — exposed as x-amz-checksum-sha256 on GET/HEAD/PUT
Object route ([...key]/route.js): ?tagging (PUT/GET/DELETE), x-amz-tagging header	Real S3 PutObjectTagging/GetObjectTagging/DeleteObjectTagging sub-resource dispatch, plus tag-at-upload-time support via the standard x-amz-tagging query-string-style header
New: s3-compat/inventory.js: buildStorageInventory, renderInventoryCsv	Org-scoped JSON/CSV export of every live object (bucket, key, size, tags, checksum, version/lock state), computed from existing org_documents records only
New: s3-compat/batchOperations.js: runBatchOperation	SET_TAGS/SET_RETENTION/SET_LEGAL_HOLD across up to 1,000 keys, implemented as a loop over the existing per-object primitives with per-key success/failure reporting — no new low-level mutation path, Object Lock/Legal Hold enforced identically to a single manual request
New: s3-compat/analytics.js: computeS3BucketAnalytics	Real per-bucket object count/total-size/average-size/largest-objects/version-count/locked-count/legal-hold-count/tag-distribution, computed live, no materialized metrics table
New: s3-compat/policyAnalyzer.js: analyzeS3CredentialPolicies	Read-only credential-scope analysis (UNRESTRICTED_CREDENTIAL, NO_EXPIRY, BROAD_DESTRUCTIVE_SCOPE, BROAD_UNSCOPED findings) over the existing s3_credentials collection via listS3Credentials — zero writes; unusedCredentials is explicitly null (not []), since no lastUsedAt field exists to support that claim
Fixed: pinningProviders/pinata.js: authHeaders/resolveJwt	Detects a usable Pinata credential by shape (a JWT is three dot-separated base64url segments starting with "eyJ") across PINATA_JWT, PINATA_SECRET_API_KEY, and PINATA_API_KEY rather than one hardcoded variable name — this environment's real credential was a scoped-key JWT stored under PINATA_SECRET_API_KEY, previously unrecognized
Test coverage	test/s3-compat-expansion.test.mjs (9 tests: checksum correctness, tag validation limits, mixed-success batch jobs, Object Lock bypass rejection inside a batch job, org-scoped inventory isolation, per-bucket analytics correctness, policy-analyzer read-only guarantee) — all passing, plus the full pre-existing 57-test S3-compat suite (store/capabilities/empty-folder/sigv4) re-confirmed clean

Explicitly out of scope for this pass, per the SOW's own audit-first instruction not to build a feature merely because AWS has it: event notifications/webhooks (deferred at the time, then built in October 2026: signed AWS-format webhook delivery for uploads, deletes and lifecycle expiry, with retry and backoff, a dead-letter list and manual redelivery, set up by an owner or admin in the S3 management screen); MFA/step-up authentication (Object Lock/Legal Hold already provide real, server-enforced protection against the highest-risk destructive action); Block Public Access (confirmed not applicable — the S3-compat auth layer has exactly three authenticated paths and a hard rejection otherwise, so there is no unauthenticated read path to block). Full writeup: docs/aws-s3-feature-expansion-report.md.

SECTION 29

Evidence Graph & Digital Twin Reference — Business Events, Passports, and Dependency-Graph Simulation (September 2026)

FILE / ROUTE	PURPOSE
New: src/lib/businessEvents.js: createBusinessEvent, getBusinessEvent, listBusinessEvents, addBusinessEventRelationship, getBusinessEventTimeline	References an existing invoice/PO/PR/AI-action-request by {subjectType, subjectId}, never copies it; department-scoped (or org-manager-only for AI-action subjects with no resolvable department) via the same canAccessDepartment/canManageOrg gates every other Business Operations record uses; timeline merges real org_activity entries from the event, its subject, and every linked relationship target
New: src/lib/businessEventExplain.js: explainBusinessEvent	Why? explainability — resolves each relationship target into INCLUDED/RESTRICTED/REFERENCED/UNAVAILABLE, evaluates one real, disclosed amount-threshold rule, and surfaces only the structured fields an AI action request actually stores (proposedAction, riskLevel, requestedContextSummary) — never internal reasoning, since none is persisted
New: src/lib/businessEventPassport.js: buildBusinessEventPassport, verifyBusinessEventPassport, renderBusinessEventPassportPdf	Reuses evidenceExporter.js's exact canonicalizeForExport hashing convention for the manifest hash; verification independently recomputes the hash and live-reverifies the org's audit chain, returning VERIFIED/INVALID/INCOMPLETE/UNKNOWN
New: src/lib/businessEventSimulate.js: simulateBusinessEventDecision	Re-derives PO_TRANSITIONS/PR_TRANSITIONS legality and the real canManageOrg/canAccessDepartment authorization checks without ever importing transitionPurchaseOrder/transitionPurchaseRequest/reviewAiAction — zero write path by construction
New: src/lib/digitalTwin.js: resolveDependents, traverseDependencyGraph	Computes the dependency graph on demand from real foreign keys already on real records (supplierId on a PO, departmentId everywhere, projectId on tasks/deals, assigneeEmail on tasks) — depth-bounded (default 3) breadth-first traversal, RESTRICTED nodes included but never traversed further
New: src/lib/digitalTwinSimulate.js: simulateDigitalTwinScenario	SUPPLIER_UNAVAILABLE, EMPLOYEE_ACCESS_REMOVED, PROJECT_DELAYED, WAREHOUSE_UNAVAILABLE — each traverses the real dependency graph and reports a numeric consequence only when a real stored field backs it (e.g. a task's own dueDate); every other effect is explicit UNKNOWN with a stated reason
API	/api/orgs/business-events/** (create/list/detail/relationships/why/passport/verify/simulate), POST /api/orgs/digital-twin/simulate — all session-authenticated via the existing requireMembership pattern
Test coverage	47 tests across test/business-events.test.mjs, test/business-event-passport.test.mjs, test/business-event-simulate.test.mjs; 9 tests in test/digital-twin.test.mjs — all passing against the real database, zero regressions to document-permissions.test.mjs or trust-health-v2.test.mjs

A real permission gap found and fixed during development. stockLevels rows have no departmentId of their own (see inventory.js) — the generic per-dependent-doc permission check that correctly gates every other entity type in digitalTwin.js would have silently defaulted a stockLevel row to visible regardless of the caller's actual department access, since there was no departmentId field on the doc itself to check. Fixed by gating simulateWarehouseUnavailable() on the WAREHOUSE entity's own departmentId before any traversal begins, rather than relying on the generic per-dependent check; documented directly on resolveDependents() so a future caller doesn't reintroduce the same gap by exposing a raw "traverse anything" endpoint without the same starting-point check.

Cross-organization Digital Twins were evaluated in a dedicated feasibility pass (docs/digital-twin-cross-org-privacy-feasibility.md) covering zero-knowledge proofs, secure multiparty computation, trusted execution, cryptographic commitments, and a scoped purpose-bound API — recommendation is to defer until a real, named cross-org query use case exists, reusing the pre-existing orgTrustRelationships collection and the existing audit chain when that need arrives, rather than building ZK/MPC/TEE infrastructure speculatively. Twin snapshots for reproducibility, a graph-visualization UI, and natural-language scenario creation are likewise deferred. Full writeup: docs/evidence-graph-business-event-layer-report.md, docs/digital-twin-simulation-layer-report.md, docs/digital-twin-cross-org-privacy-feasibility.md.

SECTION

Modular Enterprise Adoption Layer Reference — DirectSync, Data Room Templates, Cloud Backup Scheduler & What-If Studio (September 2026)

Preceded by a published, mandatory capability audit (docs/MODULAR_ADOPTION_CAPABILITY_AUDIT.md) classifying every proposed capability across all four features as already-implemented, reusable, a genuine gap, or requiring an architectural decision — captured here so the audit discipline itself is traceable, not just its outcome.

FILE / ROUTE	PURPOSE
New: inaya-desktop/src-tauri/src/directsync.rs: SyncStateDb, SyncEngine::sync_one_file, ::handle_rename, ::handle_delete, start_watching_folder	SQLite-backed folders/sync_state tables; SHA-256 change detection requiring BOTH size and hash to match before skipping a file (never trusts m coalesces duplicate OS filesystem events and pairs a rename's from/to paths within the debounce window (falls back to a clean delete+re-uploa them). handle_rename repoints local state AND physically relocates the remote object (PUT new key, best-effort DELETE old key) — the first vers real end-to-end test caught. handle_delete only marks the row LOCALLY_DELETED; the remote object is never touched by construction.
New: inaya-desktop/src-tauri/Cargo.toml dependency	inaya-drive-core = { path = "../../inaya-drive-core" } — DirectSync's upload client is the same S3Client/sigv4 signer inaya-drive-helper and inaya- against /api/s3, the third real consumer of that shared crate, not a fourth implementation. Also: rusqlite (bundled), notify-debouncer-full, sha2, h
Modified: inaya-desktop/src-tauri/src/lib.rs	10 new #[tauri::command]s (directsync_add_folder/_remove_folder/_pause_folder/_resume_folder/_list_folders/_list_queue/_retry_failed/_store_credential/_credential_configu each gated by the existing verify_trusted_origin() idiom. DirectSyncState restores its keyring-stored credential and resumes every enabled folder' recovery — SOW §6.6); all watcher threads are stopped on tray Quit, mirroring the existing DriveState pattern exactly.
New: src/lib/dataRoomTemplates.js: BUILTIN_TEMPLATES, createDataRoomTemplate, cloneBuiltinTemplate, createRoomFromTemplate	Four built-in templates with exact section lists from the SOW's own §7.2; createRoomFromTemplate() bridges into external-data-room.js's existin (not forked) with optional templateId/sections/ndaRequired/ndaText params, all additive and backward compatible.
Modified: src/lib/external-data-room.js: createDataRoom, addDocumentToRoom, acceptRoomNda, listRoomDocuments	ROOM_TYPES extended with "legal"; addDocumentToRoom() validates an optional section against room.sections; acceptRoomNda() is idempote access event; listRoomDocuments() withholds documents (returning only ndaRequired/ndaText) until session.ndaAcceptedAt is set.
New: src/lib/dataRoomEvidence.js: exportDataRoomEvidence	Reuses evidenceExporter.js's canonicalizeForExport + sha256 convention; owner/admin only; self-audits via an EVIDENCE_EXPORTED activity ent
New: src/lib/cloudBackupScheduler.js: createBackupSchedule, runBackupJob, diffAgainstLastSync, computeHealthStatus, findDueSchedules	Orchestrates inaya-migration-agent's real createAwsSource/createAzureSource/createGcsSource adapters and runMigration() engine via a file: d diffAgainstLastSync() compares each listed object's real size AND etag (never timestamp alone) against a backupObjectState Mongo collection, p through runMigration()'s own objectKeys parameter with a duck-typed shim manifest ({isDone, record, summary}) whose isDone() can honestly a already decided what changed. Destination writes call s3-compat/store.js's putS3Object/headS3Object directly, in-process, rather than the packa createInayaDestination(). computeHealthStatus() returns HEALTHY/WARNING/DEGRADED/FAILED/PAUSED/UNKNOWN from consecutiveFailures schedule's own interval.
New: src/lib/backupCryptoAndCredentials.js: storeBackupCredential, resolveBackupCredential, revokeBackupCredential	AES-256-GCM envelope encryption (iv(12)+authTag(16)+ciphertext, one base64 string) under BACKUP_ENCRYPTION_KEY — a key dedicated sol established one-key-per-secret-class rule. resolveBackupCredential() is the only decrypt path, called exclusively from inside a scheduled run, neve
Modified: src/lib/digitalTwinSimulate.js: simulateDigitalTwinScenario, listDigitalTwinSimulations	Added MODEL_VERSION/RULES_VERSION constants and an independently-recomputable integrityHash (createHash("sha256").update(canonical simulationId/runAt/runByEmail attached to the returned simulation; listDigitalTwinSimulations() reads DIGITAL_TWIN_SIMULATION org-activity e isolated. The simulation engine itself is unchanged — this is the What-If Studio's only new backend surface.
API	GET/POST /api/orgs/backup-credentials, DELETE .../[credentialId]; GET/POST /api/orgs/backup-schedules, PATCH/DELETE .../[scheduleId], POST ... gated GET /api/cron/cloud-backup-run (vercel.json, */15 * * * *); GET/POST /api/orgs/data-room-templates, PATCH .../[templateId]; GET /api/orgs POST /api/data-room-access/nda; GET (extended) /api/orgs/digital-twin/simulate — all session-authenticated via the existing requireMembersh
New UI: src/components/business/CloudBackupSchedulerView.js, DirectSyncView.js, WhatIfStudioView.js; extended DataRoomsView.js	Self-contained-view convention (local api() helper, no shared fetch wrapper) matching every other Business Workspace view. DirectSyncView call directly — the same pattern the app's existing pending-approvals/security-feed initialization scripts already use — and renders an honest "requi when window.__TAURI__ is absent.
Test coverage	test/data-room-templates.test.mjs: 8 tests (template creation/permission, builtin cloning, room-from-template, template-edit-doesn't-touch-exis rejection, NDA gate, evidence export hash+audit+permission), zero regressions to test/external-data-room.test.mjs's 8 tests. test/cloud-backup- including two using node:test's mock.module() (--experimental-test-module-mocks) to replace @inaya-network/migration-agent/src/adapters/a in-memory source BEFORE cloudBackupScheduler.js's first import — the same mocking idiom test/mfa.test.mjs already established — so runBac manifest/runMigration()/verify/persist path is genuinely exercised end-to-end, not just its pieces in isolation. test/digital-twin.test.mjs: +2 tests (i recomputable, scenario history ordering/isolation), 11 total. directsync.rs: 7 #[test] unit tests (relative-key computation, duplicate-safety logic, ha transitions, rename repointing) plus 1 #[ignore] real end-to-end test, run manually with DIRECTSYNC_TEST_ENDPOINT/_ACCESS_KEY_ID/_SECRET environment variables (never hardcoded) against a genuinely running dev server and a real, disposable S3-compat credential.

Two real bugs found and fixed during development. cloudBackupScheduler.js's runBackupJob() originally set both objectsSeen and objectsChanged from the same changedKeys.length value, so a no-op run (0 changes) reported objectsSeen: 0 even though the source was genuinely listed and examined — caught by the duplicate-safety test asserting objectsSeen on a second, unchanged run; fixed by returning seenKeys.size separately from diffAgainstLastSync() and threading it through as its own field, independent of objectsChanged. directsync.rs's handle_rename() originally only executed the UPDATE sync_state SET local_path/destination_key SQL statement — it never issued a PUT under the new key or a DELETE of the old one, so the object in Inaya silently stayed under its pre-rename key while local state claimed otherwise. The real end-to-end test caught this directly (head_object on the new key returned None); fixed by reading the unchanged local bytes at the new path and performing the actual PUT+DELETE relocation, logged via log::warn! on a best-effort DELETE failure rather than failing the whole rename.

Built later (October 2026): DirectSync Share, a time-limited download link for any synced file, signed on the machine as a standard presigned URL and verified by the server. Deliberately not built in this pass: a real-action handoff from a What-If simulation into the existing PENDING_APPROVAL flow (no concrete simulation-to-action use case has been proposed yet — the existing controlled-execution/approval machinery would be reused unchanged when one is); and an OS-level background service for DirectSync independent of inaya-desktop being open (it runs inside that existing process via the existing tray/minimize-to-background mechanism, not as a fifth standalone native service with its own installer). Full writeup: docs/directsync-report.md, docs/cloud-backup-scheduler-report.md, docs/MODULAR_ADOPTION_CAPABILITY_AUDIT.md.

SECTION

IBM Cloud VPC Storage Expansion & terraform-provider-inaya Reference (September 2026)

Preceded by a published, mandatory Phase 0 capability audit (docs/IBM_VPC_STORAGE_CAPABILITY_AUDIT.md) whose single governing finding — Inaya has no compute/VM/hypervisor runtime of any kind — is cited by name in every "physicalCapability" field the code below writes, not just asserted in prose.

FILE / ROUTE	PURPOSE
New: src/lib/storageResources.js: RESOURCE_TYPES, createStorageResource, listStorageResources, getStorageResource, expandStorageResourceCapacity, deleteStorageResource, attachVolume, detachVolume, addMountTarget, removeMountTarget, normalizeTags, matchesSelector	Every resource insert calls ensureS3Bucket() so its actual bytes live in the already-tested S3-compat storage path — this file is metadata/control-plane only, never a second storage backend. PHYSICAL_CAPABILITY constant maps volume -> "logical_only_no_compute_attach" and fileShare -> "logical_only_no_nfs_server", written once at creation and never mutated. attachVolume()/detachVolume() reject a conflicting second attach (409) rather than silently overwriting attachedTo. expandStorageResourceCapacity() rejects newCapacityGB <= current with a 400, never a silent no-op. normalizeTags() reuses s3-compat/store.js's own limits (10 tags, 128-char keys, 256-char values) generalized across resource types via one function, not copy-pasted per type.
New: src/lib/storageSnapshots.js: createSnapshot, listSnapshots, getSnapshot, restoreSnapshot, deleteSnapshot, createConsistencyGroup, listConsistencyGroups, copySnapshotToResource, shareSnapshot, revokeSnapshotGrant, resolveSnapshotGrant, captureManifest (internal)	captureManifest() enables bucket Versioning if not already on, then maps listS3Objects()'s own returned docs directly (doc.filename/doc.versionId/doc.sizeBytes) into a manifest — no second per-key headS3Object() round trip, and no bytes copied at capture time. restoreSnapshot()/copySnapshotToResource() both call getS3ObjectBody() with the manifest's recorded versionId, then putS3Object() to write it back — real copy-forward, the same technique real S3 tooling uses. createConsistencyGroup() loops createSnapshot() sequentially and stores consistencyBoundary: "SEQUENTIAL_NOT_ATOMIC" verbatim on every group record. resolveSnapshotGrant() is the one read path a recipient org can use — returns null (not an error) on wrong recipientOrgId, expiresAt in the past, or a revokedAt already set, so a caller can't distinguish "doesn't exist" from "exists but you can't see it".
New: src/lib/storageBackupPolicies.js: createBackupPolicy, listBackupPolicies, setBackupPolicyEnabled, getBackupPolicy, deleteBackupPolicy, createBackupPlan, listBackupPlans, getBackupPlan, deleteBackupPlan, runBackupPolicyPlan, listBackupJobs, findDueBackupPlans, getPlanHealth, enforceRetention (internal)	computeHealthStatus() thresholds (>= 3 consecutive failures = FAILED, > 0 = DEGRADED, no run = UNKNOWN, stale beyond 3x the plan's own interval = WARNING) are copy-identical to cloudBackupScheduler.js's, imported as a design decision for operator consistency, not coincidence. runBackupPolicyPlan() filters storageResources by matchesSelector(r.tags, policy.tagSelector), snapshots each match via createSnapshot() with a synthetic {role:"owner"} membership (explicitly commented as never a substitute for a real end user's membership — the caller that reaches this function has already authorized the action), then calls enforceRetention() per resource which deletes exactly snapshotIdsInOrder.slice(retentionCount) — every deletion logged via logOrgActivity, never a silent bulk delete. deleteBackupPolicy()/deleteBackupPlan() were added specifically to support terraform-provider-inaya's Delete() — both collections' deletedAt field existed in the schema from this file's original version with nothing that ever set it; findDueBackupPlans() and runBackupPolicyPlan() already filtered on deletedAt: null before either setter existed. getBackupPolicy()/getBackupPlan() (single-item reads) were added for the same reason — only list functions existed before.
New: src/app/api/orgs/storage/**/route.js (16 routes) + src/app/api/cron/storage-backup-run/route.js	Session-cookie auth via requireMembership(), the Business Workspace UI's own path. vercel.json cron entry: {"path": "/api/cron/storage-backup-run", "schedule": "0 * * * *"}.
New: src/app/api/public/v1/storage/**/route.js (8 routes: resources, resources/[resourceId], snapshots, snapshots/[snapshotId], backup-policies, backup-policies/[policyId], backup-plans, backup-plans/[planId])	requireApiKey()-authenticated — a synthetic {role:"owner"} membership resolved from the bearer key itself, so orgId can never be supplied or overridden by the caller (identical guarantee to the pre-existing public/v1/evidence route). Built as a parallel namespace rather than adding API-key support to the existing orgs/storage/** routes, keeping the session-cookie and bearer-key auth paths fully separate. This is the surface terraform-provider-inaya's internal/client/client.go actually calls.

FILE / ROUTE	PURPOSE
New: terraform-provider-inaya/ (Go module github.com/Talhawaqas/terraform-provider-inaya, terraform-plugin-framework v1.19.0)	internal/client/client.go: ~250-line hand-rolled HTTP client, one method per route, apiError type mirrors every route's own {error} JSON shape. internal/provider/provider.go: endpoint/api_key config attributes, falling back to INAYA_ENDPOINT/INAYA_API_KEY env vars. internal/provider/resource_storage_resource.go, resource_snapshot.go, resource_backup_policy.go, resource_backup_plan.go: one file per Terraform resource type, each with its own tfsdk model struct and RequiresReplace plan modifiers on every attribute without a real backend Update path. internal/provider/convert.go: shared *FromAPI() converters. Every Create() re-fetches via the corresponding Get() before writing state (see the bug note below) rather than trusting the create endpoint's own return shape.
Test coverage	test/storage-resources.test.mjs: 12 tests (org isolation, tag/type filtering, capacity increase-only enforcement, real reservation-conflict rejection on a second attach, mount targets as honest bookkeeping, unknown-type rejection). test/storage-snapshots.test.mjs: 11 tests (real incremental manifest + independently recomputable integrity hash, auto-versioning-enable, real copy-forward restore of genuinely overwritten content, no-manifest-entries honest no-op, soft-delete blocks restore, consistency group's non-atomic boundary and 2-resource minimum, cross-region copy moving real bytes, cross-org grant/resolve/revoke failing closed on wrong org/revocation/expiry). test/storage-backup-policies.test.mjs: 9 tests (tag-selector validation, pause/resume blocking a run, frequency/retention validation, selector-scoped execution, retention deleting exactly the right count with every deletion audited, health-status thresholds matching cloudBackupScheduler.js, due-plan filtering excluding paused policies, deleteBackupPolicy/deleteBackupPlan soft-delete plus their downstream effect on findDueBackupPlans, org isolation). test/digital-twin-storage.test.mjs: 4 tests (STORAGE_RESOURCE dependent-resolver disclosure without content leakage, STORAGE_RESOURCE_UNAVAILABLE and BACKUP_POLICY_DISABLED scenario correctness, and a read-only-by-construction proof that neither new scenario mutates any real record it reads). The Terraform provider itself: no automated Go test suite (a resource.Test-based acceptance suite is a reasonable next step), but its full create/read/update/delete cycle was run manually against a real local dev server with real MongoDB-backed state — all four resource types created in one config with real cross-resource references, a no-op terraform plan confirming zero drift, an in-place capacity_gb increase (0 to add, 1 to change), a capacity decrease correctly surfaced as a Terraform-level error rather than silently ignored, and a full terraform destroy.

A real permission-model decision: STORAGE_RESOURCE is a bespoke Digital Twin resolver, not a generic table entry. digitalTwin.js's DEPENDENT_RESOLVERS table's default behavior — no departmentId means visible to everyone — is correct for department-scoped record types, but storage resources are org-wide, gated by the distinct canAccessStorage permission. Using the generic table entry for STORAGE_RESOURCE would have been a real permission/data leak; a dedicated resolveDependents() special-case block was written instead, importing canAccessStorage directly.

A real bug found during the Terraform provider's own end-to-end testing. createBackupPlan()'s and createSnapshot()'s success return objects don't include every field their own stored documents carry — health (computed only by listBackupPlans()/getBackupPlan()) and createdAt (present on the stored doc but omitted from createSnapshot()'s explicit return literal) were both missing. terraform-provider-inaya's Create() originally wrote Terraform state directly from those create responses, so inaya_backup_plan.health and inaya_snapshot.created_at came back as empty strings immediately after terraform apply, self-correcting only on the next terraform plan/refresh. Fixed by having every resource's Create() call the matching Get() before Set()-ing state, for all four resource types — not just the two where the bug was first observed, since the same create/read field-shape mismatch risk existed for storageResources.js and storageBackupPolicies.js's policy object too, even though those two happened not to actually diverge.

Deliberately not built: fast/accelerated restore (no backend primitive exists to make a restore faster than the normal copy-forward path above); real physical block-volume attach or real multi-client NFS mounting (structurally impossible without a compute layer that doesn't exist — see the audit's central finding); context-based/conditional access restrictions (orgGates.js is pure role + department + explicit assignment; no concrete use case for time-of-day/network-context/device-context/risk-based access has been identified); a custom Terraform provider was itself originally deferred as "a separate, multi-week Go effort, out of scope for this pass" and has now been built, so this line is retained only as a record of the original honest deferral; live interop validation against real IBM Cloud Object Storage (blocked on IBM Cloud not accepting the available payment method during account creation — the exact ibmcloud CLI sequence to obtain a COS instance, HMAC access key ID/secret, and endpoint is documented and ready to run once an account exists); and publishing terraform-provider-inaya to the Terraform Registry itself (requires a public GitHub repo under the terraform-provider-<name> naming convention, GPG-signed GoReleaser releases, and registry approval — not attempted this pass). Full writeup: docs/ibm-vpc-storage-expansion-report.md, docs/IBM_VPC_STORAGE_CAPABILITY_AUDIT.md, terraform-provider-inaya/README.md.

SECTION

Official Documentation Platform Reference — inayanetwork.com/docs (September 2026)

Preceded by a published, mandatory Phase 0 audit (docs/audit/documentation-inventory.md) and a Phase 1 information-architecture doc (docs/architecture/information-architecture.md) — both cited by file path throughout the code below, not just in prose.

FILE / ROUTE	PURPOSE
New: src/lib/docsContent.js: loadAllDocs, getDocBySlug, listDocsByProduct, listAllProducts, buildSearchIndex, resolveRelatedDocs, extractHeadings/slugifyHeading (internal)	Server-only (node:fs), reads content/docs/**/*.md via gray-matter. loadAllDocs() throws on any of REQUIRED_FIELDS (slug/title/description/product/category/contentType/status) missing, an invalid status outside VALID_STATUSES, or a duplicate slug — a hard failure, not a best-effort skip. extractHeadings() derives the TOC from real ## / ### lines via regex, never hand-maintained separately. buildSearchIndex() deliberately omits full page content from its return shape (verified by test/docs-content.test.mjs) so the client-side search bundle never ships full markdown bodies.
New: src/lib/docsApiReference.js, docsSdkReference.js, docsCliReference.js	Plain exported arrays (API_ENDPOINTS, SDK_PACKAGES, CLI_TOOLS), each entry hand-verified against the real route file / package.json+README / bin script during the Phase 0 audit. No generation step reads the live route files yet — this is the one documented, deferred drift-checking gap.
New: src/app/docs/layout.js, page.js, products/[slug]/page.js, developer/[slug]/page.js, api/page.js, api/[slug]/page.js, sdk/page.js, sdk/[slug]/page.js, cli/page.js, cli/[slug]/page.js, search/page.js, release-notes/page.js	generateStaticParams() on every dynamic route pre-renders all real slugs at build time (confirmed in the build output as ● SSG entries, not ƒ dynamic). Each detail page calls notFound() for an unknown slug rather than rendering an empty shell.
New: src/components/docs/DocsShell.js, MarkdownRenderer.js, Breadcrumbs.js, TableOfContents.js, StatusBadge.js, RelatedDocs.js, DocsSearchClient.js	DocsShell scopes the dark/light theme to a `dark` class on its own wrapper div (never document root), reading/writing a inaya-docs-theme localStorage key inside try/catch per this codebase's established browser-storage discipline. MarkdownRenderer wraps the already-installed react-markdown/remark-gfm (previously used only for the AI chat widget) with a CodeBlock component adding a real copy-to-clipboard button and per-language label. DocsSearchClient ranks title matches (score 10) above tag (6), description (4), and heading (3) matches — a real, if simple, relevance ordering, not a linear scan returned in file order.
New: scripts/docs/generate-openapi.mjs	buildOperation() auto-derives path parameters from each endpoint's own {placeholder} syntax in docsApiReference.js's path field (pathParams()), separately maps GET query parameters (queryParams()) and POST/PATCH body parameters into a requestBody schema (requestBodySchema()) from the same params array's free-text `in` field. Every operation declares security: [{bearerAuth: []}] and a standard 401/403/404 response set. npm run docs:openapi regenerates public/openapi.json on demand; it is not wired into the Next.js build itself, so a docsApiReference.js edit requires a manual regenerate today (a real, documented gap, not automatic sync).
New: src/app/docs/release-notes/page.js	Imports ROADMAP_STAGES/STATUS_LABELS directly from src/lib/saasRoadmap.js — sorts by stage number descending, maps each stage's own status string (LIVE/TESTNET/BETA/PLANNED) through STATUS_TO_DOCS_STATUS into the same StatusBadge component every other /docs page uses, and renders each stage's existing features/securityStatement/notes fields verbatim rather than re-deriving or summarizing them.
New: src/lib/rag/sources/docsSources.js: docsPlatformSources(), walkContentDocs() (internal)	Walks content/docs/ the same way docsContent.js's loader does (a small, deliberate duplication rather than a shared cross-module import, since the two have different failure semantics — the loader throws on bad frontmatter, the RAG adapter should not take down ingestion for one malformed page). Prepends a synthetic `# ${title}` line before calling chunkMarkdownByHeading(), since content/docs/**/*.md bodies start at ## Overview with no H1 of their own, and chunkMarkdownByHeading() falls back to sourceId as the title when no H1 is found.

FILE / ROUTE	PURPOSE
Test coverage	test/docs-content.test.mjs (9 tests): unique slugs, every required field + valid status present across every real file, getDocBySlug resolution including the null case, every relatedDocs entry resolving to a real page, heading extraction, search-index shape (including the explicit content-leakage negative assertion), product grouping, and a gray-matter parse smoke test independent of the loader. test/docs-openapi.test.mjs (1 test): runs the real generator via child_process.execSync, then asserts the spec's path set exactly equals API_ENDPOINTS' paths, every declared method has a security block, and every {placeholder} in a path has a matching path parameter — a genuine regression guard against the spec silently drifting from the rendered reference pages, not just a smoke test that the generator doesn't crash.

A pre-existing, unrelated production bug found and fixed during this pass. `vercel ls`/`vercel inspect --logs` showed every Production deployment failing for the prior ~20 hours with `Module not found: Can't resolve '@aws-sdk/client-s3'` / `'@azure/storage-blob'`, traced to `../inaya-migration-agent/src/adapters/{aws,azure}.js` — a `file:../inaya-migration-agent` dependency whose own `package.json` dependencies were never installed by Vercel, since Vercel's install step only ever runs inside `inaya-network-dapp`. It worked locally purely by accident (that directory's `node_modules` had been populated by hand at some earlier point). Fixed with a `postinstall` script (`npm install --prefix ../inaya-migration-agent --omit=dev --no-audit --no-fund || echo ...`, non-fatal by design so a checkout without that sibling directory still installs cleanly) — verified by moving the local `inaya-migration-agent/node_modules` aside to simulate Vercel's fresh clone, confirming the hook re-populates it, and running a full `npm run build` before shipping, not just assuming the fix worked.

Deliberately not built: API/SDK/CLI drift detection against the live route files/package exports (the reference data is accurate as of this audit but nothing re-verifies it on a schedule); a Tutorials/Solutions/FAQ-as-a-system content type; wiring generate-openapi.mjs into the Next.js build pipeline itself; a full CI validation/accessibility/SEO test suite; an admin/governance interface for this content; and an API playground. Full writeup: docs/official-documentation-platform-report.md.

SECTION 30

Mainframe & Legacy Data Access Reference (September 2026)

> Phase 0 audit: docs/MAINFRAME_DATA_ACCESS_CAPABILITY_AUDIT.md. Full report: docs/mainframe-legacy-data-access-report.md.

FILE / ROUTE	PURPOSE
src/lib/legacyDataAccess/connectorRegistry.js	Connector interface + registry.
src/lib/legacyDataAccess/connectors/relational.js	The real node:sqlite reference connector.
src/lib/legacyDataAccess/connectors/rmsOpenVms.js	Real RMS/OpenVMS connector — SSH + DCL against a genuine VSI OpenVMS x86-64 instance; metadata parsed from OpenVMS's own FDL output.
src/lib/legacyDataAccess/dataSources.js, metadata.js, credentials.js	Data-source registry/health, metadata import with schema versioning, encrypted credential envelopes.
src/lib/legacyDataAccess/sqlGateway.js	Real SQL AST parser, authorization, execution, audit.
src/app/api/orgs/data-sources/**	Session-authenticated admin API.
src/app/api/public/v1/data-sources/**	API-key-authenticated external API (same library code).
src/components/business/DataSourcesView.js, SqlConsoleView.js	Admin UI.
jdbc-driver/ (Java 11, Maven shaded jar)	Real java.sql.Driver, jdbc:inaya:http://host/<dataSourceId>.
odbc-driver/inayaodbc.dll, .def, src/*.c	Real compiled Win32 ODBC driver: driver.c, http.c (WinHTTP), json.c (hand-written parser). Registration and connection through the real Windows ODBC Driver Manager now proven; query execution through that path still crashes, isolated to a fault inside Microsoft's own odbc32.dll.
Test coverage	test/legacy-data-access.test.mjs (11), test/rms-openvms-connector.test.mjs (7, live against the real OpenVMS instance), jdbc-driver integration (7), odbc-driver/test/direct_test.c (19).

SECTION 31

Sovereign NAS Reference (September 2026)

Deployment profile, support matrix and recovery runbook: docs/nas-runbook.md. Full report: docs/sovereign-nas-report.md.

FILE / ROUTE	PURPOSE
src/lib/nas/appliance/inaya-nas-agent.py	Appliance-side agent: validated JSON ops, path containment, versioned self-update with rollback.
src/lib/nas/* (25 modules)	Shares, pools, snapshots, WORM, quotas, locking, recycle bin, ransomware detection, backup, replication, identity/permission sync.
src/app/api/orgs/nas/** (49 route files)	Session-authenticated console API.
src/components/business/NasManagementView.js	18-section admin console.
scripts/nas-worker.mjs, /api/cron/nas	Background worker: snapshots, threat scans, replication, backup, quota checks, lock/lockdown expiry.
Test coverage	nas-access, nas-storage, nas-security, nas-protection, nas-realclient (real Windows SMB and Linux NFS clients, real disk-failure injection).

SECTION 32

AI Security Workflow Reference (September 2026)

Full report: AI_SECURITY_IMPLEMENTATION_REPORT.md.

FILE / ROUTE	PURPOSE
src/lib/aiSecurity/policyTypes.js	Shared decision/severity/category vocabulary (DECISIONS, SEVERITIES, EVENT_CATEGORIES, POLICY_VERSION).
src/lib/aiSecurity/{promptInjection,piiDetector,policyEngine,orgPolicy,modelRegistry,events,rateLimiting,gateway,routeGuard}.js	See architecture table above.
src/lib/businessEvents.js (additive)	AI_SECURITY_CHECK event type, subject resolver, risk classifier, summary.
src/lib/orgGates.js/orgs.js (additive)	canManageAiSecurity/canAccessAiSecurity; 3 new collections (aiSecurityChecks, aiSecurityPolicies, aiModelRegistry) with indexes.
src/app/api/orgs/ai-security/{events,policy,explain/[eventId],models}/route.js	Session-authenticated API.
src/components/business/AiSecurityView.js	Activity / Model Inventory / Policy tabs.
Test coverage	23 of 23 adversarial tests (ai-security-gateway.test.mjs) plus ai-security-wiring.test.mjs (route-level, incl. a coverage guard that fails if a new model-calling route skips the gateway).

SECTION 33

Document & Invoice Automation Reference (September 2026)

> Full report: docs/document-invoice-automation-report.md. Operational runbook: docs/document-automation-runbook.md.

FILE / ROUTE	PURPOSE
src/lib/documentAutomation/{adapters,calculations,money,templateSchema,renderer,lifecycle,jobs,delivery}.js	Adapters, exact-money calculation, safe template schema, pdfkit renderer, lifecycle state machine, retry jobs, secure delivery/verification.
src/app/api/orgs/documents-automation/**	Session-authenticated admin/creation API (create, preview, generate, approve, finalize, deliver).
src/app/api/cron/document-automation	Hourly retry of failed storage/evidence, expiry of stale approvals and lapsed quotations, invoice PAID/CANCELLED sync.
next.config.mjs experimental.outputFileTracingIncludes / serverComponentsExternalPackages	Ships pdfkit and its font data (incl. the standard-fonts chunks/ helper and the Noto Arabic/Sans fonts) to every API route's serverless function (SQA-026 fix).
test/pdf-serverless-config.test.mjs	Regression guard for the SQA-026 config: pins the tracing includes and checks the installed pdfkit package still matches what they target.
Test coverage	59 checks before the push (unit 20, types 13, lifecycle 13, security 10, e2e 3) plus regression suites; two evidence-exporter tests fail only because of a pinning-provider plan limit, unrelated to this code.

SECTION 34

AI Business Operations Manager Reference (September 2026)

> Full report: docs/ai-business-operations-manager-report.md.

FILE / ROUTE	PURPOSE
src/lib/workflows/{nodes,data,engine,templates}.js	Node types, data readers (incl. data.inaya_support_tickets), execution engine, ready-made templates (incl. finance-operations-manager).
src/lib/ai-action-requests.js (reused)	PENDING_APPROVAL → APPROVED (+36h) → QUEUED → EXECUTED — the existing Controlled Actions engine, not duplicated.
src/app/api/orgs/workflows/**	Session-authenticated CRUD, publish, execute, credentials, evaluations.
src/app/api/cron/workflows	Every 5 minutes on the paid Vercel plan.
scripts/gmail-refresh-token.mjs	Local, one-time Gmail OAuth refresh-token helper — stores nothing, uses a temporary 127.0.0.1 listener.
docs/workflow-slack-app-manifest.json	A pasteable Slack app manifest, replacing manual settings click-through.
Test coverage	workflow-unit (16), workflow-security/features (19+12), workflow-acceptance (7 of 7 reference flow), workflow-routes, finance-workflow.

SECTION 35

Customer Portal & Customer Service Reference (September 2026)

> Phase 0 audit: CUSTOMER_PORTAL_CAPABILITY_AUDIT.md. Full report: docs/customer-portal-report.md.

FILE / ROUTE	PURPOSE
src/lib/support/*.js (30 files)	See architecture table.
src/lib/help.js (new)	getSupportDesk, getHelpConfig, createHelpTicket, notifySupportMailbox.
src/app/api/orgs/support/**, /upload, /attachments/[id], /export	Agent console API.
src/app/api/portal/[slug]/**	Customer portal API.
src/app/api/public/v1/support/**	API-key-authenticated external API.
src/app/api/support/inbound-email/[slug]	Resend inbound webhook (Svix-verified) — setup pending.
src/app/api/help/{config,ticket}/route.js (new)	Public config read; authenticated ticket creation, rate-limited 5/hour/user.
src/app/api/cron/support (every 5 minutes)	SLA ticking, escalation, retention.
Test coverage	support-sla, support-unit, support-core, support-integrations, support-routes, support-scanner, support-live-ai, help-support.test.mjs (7 tests for Help & Support).

SECTION 36

Identity Integration Reference (September 2026)

Phase 0 audit: IDENTITY_INTEGRATION_CAPABILITY_AUDIT.md. Full report: docs/identity-integration-report.md.

FILE / ROUTE	PURPOSE
src/lib/identity/{providers,mapping,engine,overrides,grants,revocation,scim,reconcile,msp,credentials,temporary,reviews,orphans,api,outbound}.js	See architecture table.
src/lib/identity/entraGraph.js	Optional Microsoft Graph directory pull, verified against a real Entra tenant.
src/app/api/scim/v2/**	Standard SCIM 2.0 endpoints.
src/app/api/integrations/identity/**, /webhooks/[provider]	One dispatcher (api.js) plus signed inbound webhooks.
custody-sdk/src/identityIntegration.js	Thin, stateless client for the above (InayaKernel.IdentityIntegration), for Rewst workflows/RMM scripts/small services.
ad-sync-agent/ (own package.json, own tests)	Real, standalone on-prem LDAP agent a customer runs next to their own domain controller — outbound-only, closing the prior 'Active Directory: unsupported by design' gap. Verified live against a real Windows Server domain controller; three real bugs found and fixed (tenantId derivation, AD GeneralizedTime conversion, an ldapjs unbind() race).
src/components/business/identity/** (15-tab console)	Business Workspace Identity & Access admin UI.
content/docs/identity/* (14 pages)	VERIFIED/PARTIAL/UNVERIFIED/UNSUPPORTED/FUTURE labelling per integration.

SECTION 37

AI Bookkeeper Reference (September 2026)

> Phase 0 audit: AI_BOOKKEEPER_CAPABILITY_AUDIT.md. Full report: docs/ai-bookkeeper-report.md.

FILE / ROUTE	PURPOSE
src/lib/bookkeeper/{common,db,record,settings,sources,bank,extract,documents,categorize,match,anomaly,policy,reconcile,review,inbound,insights,period,twin,workflow,assistant,api,worker}.js	Full pipeli hold prop
src/app/api/orgs/finance/bookkeeper/[[...path]]/route.js	Session-a dispatche
src/app/api/finance/bookkeeper/{ingest,whatsapp}/[sourceId]/route.js	Signed en WhatsApp
src/app/api/cron/bookkeeper	15-minute
src/components/business/{BookkeeperView,bookkeeper/*}.js	9-tab con
Evidence Graph subjects	BOOKKEE BOOKKEE
Test coverage	bookkeep bookkeep bookkeep database,

SECTION 38

Whole-Codebase Quality & Security Review Reference (September 2026, in progress)

> Full defect registry, methodology and open risks: docs/sqa/master-defect-registry.md.

FILE	PURPOSE
src/lib/bridgeMessage.js	Pure message-hashing/verification helpers, pinned against the real deployed contract by Test/BridgeMessageHashParity.test.js.
src/lib/stripeSettlement.js	Atomic per-session settlement claim with per-step progress, used by the Stripe webhook.
src/lib/s3-compat/{sigv4,multipartCompletion,objectBodyCache,etag}.js	SigV4 canonical-URI fix and presigned-URL support, idempotent multipart completion, read single-flight, real-S3-shaped ETag.
src/lib/pinningProviders/local.js	Opt-in local pinning provider for development/CI, never active on Vercel.
next.config.mjs, test/pdf-serverless-config.test.mjs	The pdfkit serverless-bundling fix and its regression guard.
test/sqa-*.test.mjs (tenant-isolation, anon-sweep, bridge, stripe, faucet, signature-freshness, node-auth, points-auth, s3-store, s3-realclient, s3-presign, local-provider, anon-limits, node-listing, interop-register)	The regression suite this review added.

SECTION 39

Managed Database Service Reference (September 2026)

> Capability audit: RDS_SAGEMAKER_DOCINT_CAPABILITY_AUDIT.md. Real, current Supabase Management API endpoints confirmed against supabase.com/docs/reference/api before any code was written.

FILE	PURPOSE
src/lib/rds/providerRegistry.js	Provider registry; only "supabase" registered this pass.
src/lib/rds/providers/supabase.js	POST/GET/DELETE /v1/projects, .../pause, .../restore, .../database/backups, .../database/backups/restore-pitr, .../database/query -- Bearer SUPABASE_ACCESS_TOKEN, read at call time only.
src/lib/rds/instances.js	Control plane: provision/list/status/start/stop/deprovision/snapshots/PITR-restore/query, owner-or-admin gated, DB password AES-256-GCM encrypted via integrationCrypto.js.
src/lib/rds/record.js	audit/link/notify -- RDS_INSTANCE Evidence Graph subject.
src/app/api/orgs/rds/instances/[[...]]/route.js (4 route files)	Session-authenticated REST surface.
src/components/business/DatabasesView.js	Business Workspace panel: provision, start/stop, delete (typed-name confirmation).
Test coverage	test/rds-instances.test.mjs -- 6/6, real database for Inaya's own records; the Supabase API itself intercepted at the fetch boundary so no real project is ever created by CI.

SECTION 40

Document Intelligence Studio Reference (September 2026)

FILE	PURPOSE
src/lib/docIntelligence/analyzers.js	6 built-in analyzers (invoice/PO/receipt/contract/classify/summarize) + custom, DRAFT->TESTING->READY->ACTIVE->DISABLED->ARCHIVED lifecycle.
src/lib/docIntelligence/extract.js	extractDeterministicGeneric() (schema-driven, requires an explicit ":"/"#" separator for string/date fields to avoid false-positive matches), extractFieldsWithAi/classifyWithAi/generateWithAi (all through support/ai.js's gatedJson), mergeFields() (grounding-capped confidence, identical discipline to bookkeeper/extract.js's mergeAi).
src/lib/docIntelligence/analyze.js	Orchestrator: scanBuffer -> storeBytes (s3-compat, own "doc-intelligence" bucket) -> extract/classify/generate -> review routing -> Evidence Graph.
src/lib/docIntelligence/review.js	Versioned corrections (result.corrections[] appended, never overwrites result.fields) -- currentFields() is the read-time projection that overlays them.
src/lib/docIntelligence/evaluate.js	precision/recall/fieldAccuracy/groundingRate/correctionRate, reported as separate named metrics.
src/app/api/orgs/doc-intelligence/{analyzers,analyze,results,review,evaluate}/[[...]]/route.js (8 route files)	Session-authenticated REST surface; member can submit, owner/admin manages the registry and review queue.
src/components/business/DocIntelligenceView.js	Business Workspace panel: submit, view fields/grounding, resolve review items.
Test coverage	test/doc-intelligence.test.mjs -- 10/10, real database, real Evidence Graph, real cross-org isolation check.

SECTION 41

AI/ML Studio Reference (September 2026)

FILE	PURPOSE
src/lib/mlStudio/catalog.js	registerCatalogEntry() VERIFIES the ref against a real source (getVirtualTableByName / getAnalyzer / a real org_documents project) before accepting it -- no dangling entries possible.
src/lib/mlStudio/dataQuality.js	createRule/runRule -- NOT_NULL/UNIQUE/RANGE/ROW_COUNT_MIN translated to SQL and run through legacyDataAccess/sqlGateway.js's EXISTING executeVirtualQuery(); REGEX honestly refused (no portable REGEXP in the reference SQLite connector).
src/lib/mlStudio/models.js	registerModel() hashes the artifact (sha256), stores it via s3-compat/store.js (same encrypted, sharded, multi-provider path as everything else), same lifecycle taxonomy as analyzers.
src/lib/mlStudio/evaluations.js	recordEvaluation() -- an arbitrary named-metrics object (up to 30 metrics), CHECKED_BY-linked to the model version.
src/lib/mlStudio/execution.js	runCodeSnippet() -- @vercel/sandbox, python3 -c / node -e, networkPolicy "deny-all" by default, env: {} (Inaya's own secrets never passed in), 2-minute hard cap, rate-limited 20/hour, owner/admin only.
src/app/api/orgs/ml-studio/{catalog,models,execute}/[[...]]/route.js (9 route files)	Session-authenticated REST surface.
src/components/business/MlStudioView.js	Business Workspace panel: model list, code-execution form with live stdout/stderr.
Test coverage	test/ml-studio.test.mjs (5/5, real SQLite fixture) + test/ml-studio-execution.test.mjs (5/5, real live Vercel Sandbox microVMs, not mocked).

A real bug this suite caught before shipping. orgs.js registered the ML_STUDIO_MODEL Evidence Graph collection under the wrong physical Mongo collection name (ml_models vs. the real ml_model_versions) -- Evidence Graph linking silently failed until the ml-studio test's own assertion caught it. Fixed and re-verified green.

SECTION 42

Secure Collaboration & Enterprise Governance Reference (October 2026)

FILE	PURPOSE
src/lib/chat/mlsServer.js	Server-side MLS (RFC 9420) group state and ciphertext relay -- the server never holds a group's decryption-capable key.
src/lib/chat/{conversations,devices,contacts,presence}.js	Conversation membership, per-device MLS client registration, contact resolution and presence, scoped to the existing org/department permission model.
src/lib/chat/transport.js	The same long-poll/SSE real-time pattern Meet's signaling layer reuses -- one real-time primitive platform-wide.
src/lib/chat/attachments.js	Encrypted attachments over the existing sharded storage backend, not a second storage path.
src/lib/notes/notes.js	Secure Notes -- end-to-end encrypted on the same client-side model and device-key registry as chat.
src/lib/sharing/{shares,policy}.js	Sharing 2.0: expiring/password-protected/watermarked links and inbound file requests, layered on the existing secure-sharing primitive.
src/lib/governance/{classification,classifyRules,dlp}.js	Document classification and data-loss-prevention rule matching, enforced at the point data would leave a scope.
src/lib/governance/{retention,lifecycle,metadata}.js	Automated retention/legal-hold policies and lifecycle transitions.
src/lib/governance/enforce.js	The single checkpoint every governed action passes through.
src/lib/governance/events.js	Journals governance decisions into the same Evidence Graph every other feature uses.
src/lib/gateway/{gateway,acl,transfers,health,modes}.js	Sovereign Gateway agent connecting a customer's own on-premises file server -- its own ACL model, transfer queue, and health reporting.

114 tracked deliverables across 26 workstreams: 49 finished and independently checked against the real database and, where there's a screen, a real browser; 35 built and tested in code but not yet proven against a real outside system (the Sovereign Gateway, the NTFS/AD folder-permission bridge, HA site replication, and Microsoft 365/Outlook integration among them); 27 partly built; 3 not built by deliberate choice (mobile chat, push notifications). The MLS library's open-source implementation has not had an independent security audit. Full ledger: docs/architecture/secure-collaboration-implementation-ledger.md.

SECTION 43

Sovereign Infrastructure Fabric Reference (October 2026)

FILE	PURPOSE
inaya-infrastructure/crates/inaya-infra-node/src/main.rs	Node daemon entry point a customer installs on their own hardware -- registers, polls for operations, reports capacity.
inaya-infrastructure/crates/inaya-infra-node/src/operations.rs	One match arm per operation (CREATE_INSTANCE, BOOT_GUEST_INSTANCE, CREATE_VOLUME, CREATE_VPC, CREATE_LOAD_BALANCER, UPDATE_LOAD_BALANCER_TARGETS, DELETE_LOAD_BALANCER).
inaya-infrastructure/crates/compute/src/lib.rs	QemuKvmProvider -- real VM create/start/stop/restart/delete, tracked by PID.
inaya-infrastructure/crates/compute/src/cloud_image.rs	Cloud image caching: downloads a real Ubuntu Server qcow2 image once per node, SHA256SUMS-verified.
inaya-infrastructure/crates/compute/src/cloud_init.rs	NoCloud user-data/meta-data/network-config seed ISO -- what actually gets a guest online with a real static IP.
inaya-infrastructure/crates/block/src/lib.rs	Real qcow2 volumes; create_volume_from_image() for copy-on-write overlay disks.
inaya-infrastructure/crates/network/src/lib.rs	LinuxBridgeNetworkProvider -- per-VPC bridge, nftables default-drop, gateway IP, NAT/masquerade.
inaya-infrastructure/crates/guest-exec/src/lib.rs	Pure-Rust SSH client (russh) for real commands/file pulls inside a guest -- general-purpose, reused by the Kubernetes work below.
inaya-infrastructure/crates/certificates/src/lib.rs	Per-node keypair issuance and signed-request auth (the Vercel-edge-constrained alternative to literal TLS mTLS).
inaya-infrastructure/crates/telemetry/src/lib.rs	Heartbeat capacity reporting; detect_tpm() and detect_confidential_compute_flags() added this round, detection only.
inaya-infrastructure/crates/loadbalancer/src/lib.rs	HaproxyLoadBalancerProvider -- renders haproxy.cfg and spawns/reloads/stops a real haproxy process.
src/lib/infrastructure/awsSigV4.js	Real SigV4 canonical-request + HMAC chain, verified against AWS's own published worked example.
src/lib/infrastructure/awsCredentials.js	Mints a real Inaya-issued key pair for use with the real aws CLI/boto3 via --endpoint-url.
src/lib/infrastructure/awsEc2Xml.js	Real EC2-shaped XML responses (botocore's EC2 model expects XML, not JSON).
src/app/api/aws-ec2/route.js	Single POST entry point dispatching on the real EC2 query protocol's Action field.
src/lib/infrastructure/loadBalancers.js	CRUD over infra_load_balancers, mirroring operations.js's existing volume/VPC pattern.
src/lib/infrastructure/k8sClusters.js	Tracks k3s clusters; retrieves the real node-token and kubeconfig over the guest-exec SSH channel.
src/components/business/InfrastructureView.js	Business Workspace console: nodes, VMs, capacity, clusters, load balancers, access keys -- Kubernetes/AWS CLI/Load Balancers tabs added this round.
terraform-provider-inaya (compute resources)	3 compute resources, intentionally separate from the pre-existing storage-only Terraform provider.
Test coverage	30 Rust tests + 52 Node.js tests (test/infrastructure-{aws-ec2,aws-sigv4,k8s,loadbalancers}.test.mjs among them), all passing.

Two real bugs found by actually running the software, not assumed from the code. NodeCapacity was serialized camelCase on one side of the wire and read snake_case on the other -- the scheduler's "most free memory" comparison silently never matched. Fixed with #[serde(rename_all = "camelCase")] in api-types. Separately, guest-exec's exec() broke its read loop on ChannelMsg::Eof, which worked by luck for short SSH responses but silently truncated longer ones (the kubeconfig) because ExitStatus can legitimately arrive after Eof for larger payloads -- fixed to only break on ChannelMsg::Close.

Proven for real: a real Ubuntu cloud image boots under QEMU/KVM and reaches the internet through the VPC's NAT rule; the real aws CLI creates and lists a real instance; a real haproxy process round-robins real connections across two real backends surviving a live reload; a real k3s server and agent come up on real guest VMs with kubectl get nodes showing both Ready. Explicitly not built: GPU passthrough (no dedicated GPU host to test against), full AWS coverage (S3/IAM/autoscaling), HTTP-aware load balancing, and TPM-backed identity or real SEV/TDX execution.

SECTION 44

Internxt-Inspired Expansion Reference — Post-Quantum Cryptography, Cleaner, Meet & Backup Reconciliation (October 2026)

A delta scope on top of the Competitive Expansion SOW above, gated by the same non-duplication discipline: a full audit of 8 existing backup-related files, the chat/device/PQC-key registries, and the desktop native-command pattern happened before any new code, recorded in docs/internxt-gap-analysis.md, docs/internxt-feature-reuse-matrix.md, and the living docs/internxt-implementation-ledger.md this whole section summarizes.

FILE / ROUTE	PURPOSE
custody-sdk/src/pqc/provider.js: generateKeyPair, encapsulate, decapsulate	Hybrid ML-KEM-768 + X25519 key establishment. ml_kem768 from @noble/post-quantum (hashes-only dependency) combined with the SDK's existing x25519 import through a hand-rolled HKDF-SHA256 -- NOT the package's own hybrid.js preset, which was found during development to hard-require a major version of @noble/curves this app's existing wallet/signing code doesn't use, a real dependency conflict npm run build itself caught. info parameter binds a domain-separation label plus both ciphertexts and both public keys, so tampering with any component changes the derived key rather than leaving it unauthenticated -- never SHA256(a+b).
custody-sdk/src/pqc/envelope.js, agileSharing.js: wrapContentKeyHybrid, wrapForRecipient, SHARING_MODE	A versioned hybrid envelope (AES-256-GCM over the derived shared secret) and an algorithm-agile extension of the SDK's own existing X25519/XChaCha sharing primitive -- LEGACY_CLASSICAL calls that unchanged function byte-for-byte; HYBRID_PQC/PQC_REQUIRED route through the new hybrid path, with PQC_REQUIRED refusing outright rather than silently downgrading.
src/lib/pqc/deviceKeys.js: registerDeviceKey, activeKeyForDevice, revokeAllKeysForDevice	pqc_device_keys collection -- public key, algorithm, status only; a key must reference an already-checked-in org_devices record. revokeAllKeysForDevice() existed from the start but was never actually called by the device-revocation workflow until this pass's Workstream E wiring (see below) -- a real, closed gap, not a new function.
inaya-desktop/src-tauri/src/cleaner.rs	scan_temporary_files, scan_duplicates (content-hash grouping, size-staged for performance), protected_paths/is_protected re-checked at the actual deletion call site (not just scan time), cleanup_selected uses the trash crate (OS recycle bin, not permanent delete). Wired through the existing build.rs COMMANDS manifest + capabilities/default.json allow-* + verify_trusted_origin pattern identical to every other command in lib.rs -- no new native-command plumbing invented.
src/lib/meet/{common,rooms,signaling}.js	Room lifecycle (meet_rooms/meet_participants), signaling (meet_signals, 24h TTL) over the exact long-poll/SSE transport pattern src/lib/chat/transport.js already established -- not a second real-time stack, since Vercel serverless cannot hold a persistent WebSocket open. Moderator authorization re-derived from the database on every call, never a client-asserted role. Media is peer-to-peer mesh, never routed through an Inaya server -- a deliberate v1 decision recorded in docs/architecture/meet-architecture-adr.md given no SFU can run on serverless infrastructure without new always-on hosting.
src/lib/chat/conversations.js: KINDS	Extended with a "meeting" value so Meet's in-call chat reuses the existing MLS conversation model -- one line, not a second message system. A meeting's conversation starts with zero invitees (the host only); participants are added via the same addParticipants() every other kind already uses as they're admitted to the room.
src/app/api/meet/rooms/** (12 route files) + src/app/meet/[token]/page.js	Dual-auth design throughout: an org session resolves identity from the cookie; a guest with no Inaya account authenticates via participantId + a hashed, single-use guestToken returned once at join time. The public /meet/[token] page works for both without branching UI logic at the route level.

FILE / ROUTE	PURPOSE
src/components/business/meet/MeetView.js	Real RTCPeerConnection mesh -- a deterministic glare-free initiator rule (the lexicographically-smaller participant id offers), getUserMedia/getDisplayMedia, and client-side PQC key resolution for admission: fetches a waiting participant's active device key and calls Pqc.wrapForRecipient for real before approving, not a stub. No TURN server configured (STUN-only) -- stated plainly as a known v1 limitation, not hidden.
src/lib/devices/devices.js: deviceAction	revoke/block now also calls pqc/deviceKeys.js's revokeAllKeysForDevice() in the same action (fire-and-forget, matching the function's existing webhook/notify calls) -- the Workstream E gap this pass closed. Also: a per-device PQC-capability badge and backup-health badge added to DevicesView.js, each one batched fetch for the whole list, not N+1 queries.
src/components/business/DeviceProtectionView.js	Consolidates four previously scattered sidebar entries (Devices, Endpoint Backup, DirectSync, Cleaner) into one tabbed surface, per the SOW's own explicit instruction not to expose confusing separate modules. Pure UI composition -- none of the four underlying views' logic moved.

Two real bugs found and fixed by actually running the software, not assumed from the code. A live verification pass against a real running server (a real organization, a real login through the production magic-link route, a real browser) found: (1) a pre-existing crash in business/page.js's Sidebar component -- a nav-item filter referenced a `membership` variable that was never actually passed into that component as a prop, throwing an uncaught ReferenceError for any user whenever a timing race left `canManage` momentarily false, with no recovery short of a page reload; (2) a Meet-specific bug where a denied camera/microphone permission and a genuine room-load failure shared one error-state variable, so any user without a camera -- or who simply declined the browser's permission prompt -- was incorrectly told the meeting had failed to open instead of being let in without video. Both fixed and re-verified in the same pass.

Honestly scoped as still open, because no automated or sandboxed pass can close it: a real two-person video call between two real browsers with real cameras (the engineering sandbox used for this pass blocks camera/microphone access entirely, by design), and a real phone running the mobile app's Meet join screen (built: a web-view around the same /meet/[token] link; camera/microphone permissions added to app.json but need a real native build, not just an over-the-air update, to actually take effect). 127/127 SDK cryptography tests (including official ML-KEM test vectors and adversarial downgrade/tamper cases), 9/9 Cleaner filesystem tests, and 28 server-side Meet/device tests are passing; the full backup-system audit, consolidation, and PQC-applicability finding are written up in docs/internxt-implementation-ledger.md, which states plainly, row by row, what's VERIFIED versus PARTIAL versus DEFERRED versus genuinely NOT_APPLICABLE given this codebase's own architecture -- never rounding up.